

Deep Residual Learning for Image Recognition

Kaiming He Xiangyu Zhang Shaoqing Ren Jian Sun
Microsoft Research
{kahe, v-xiangz, v-shren, jiansun}@microsoft.com

Abstract

Deeper neural networks are more difficult to train. We present a residual learning framework to ease the training of networks that are substantially deeper than those used previously. We explicitly reformulate the layers as learning residual functions with reference to the layer inputs, instead of learning unreferenced functions. We provide comprehensive empirical evidence showing that these residual networks are easier to optimize, and can gain accuracy from considerably increased depth. On the ImageNet dataset we evaluate residual nets with a depth of up to 152 layers—8× deeper than VGG nets [41] but still having lower complexity. An ensemble of these residual nets achieves 3.57% error on the ImageNet test set. This result won the 1st place on the ILSVRC 2015 classification task. We also present analysis on CIFAR-10 with 100 and 1000 layers.

The depth of representations is of central importance for many visual recognition tasks. Solely due to our extremely deep representations, we obtain a 28% relative improvement on the COCO object detection dataset. Deep residual nets are foundations of our submissions to ILSVRC & COCO 2015 competitions¹, where we also won the 1st places on the tasks of ImageNet detection, ImageNet localization, COCO detection, and COCO segmentation.

1. Introduction

Deep convolutional neural networks [22, 21] have led to a series of breakthroughs for image classification [21, 50, 40]. Deep networks naturally integrate low/mid/high-level features [50] and classifiers in an end-to-end multi-layer fashion, and the “levels” of features can be enriched by the number of stacked layers (depth). Recent evidence [41, 44] reveals that network depth is of crucial importance, and the leading results [41, 44, 13, 16] on the challenging ImageNet dataset [36] all exploit “very deep” [41] models, with a depth of sixteen [41] to thirty [16]. Many other non-trivial visual recognition tasks [8, 12, 7, 32, 27] have also

¹<http://image-net.org/challenges/LSVRC/2015/> and <http://mscoco.org/dataset/#detections-challenge2015>.

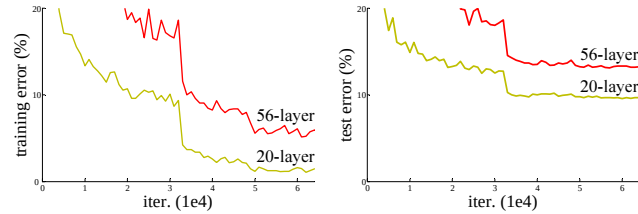


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer “plain” networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

greatly benefited from very deep models.

Driven by the significance of depth, a question arises: *Is learning better networks as easy as stacking more layers?* An obstacle to answering this question was the notorious problem of vanishing/exploding gradients [1, 9], which hamper convergence from the beginning. This problem, however, has been largely addressed by normalized initialization [23, 9, 37, 13] and intermediate normalization layers [16], which enable networks with tens of layers to start converging for stochastic gradient descent (SGD) with back-propagation [22].

When deeper networks are able to start converging, a degradation problem has been exposed: with the network depth increasing, accuracy gets saturated (which might be unsurprising) and then degrades rapidly. Unexpectedly, such degradation is *not caused by overfitting*, and adding more layers to a suitably deep model leads to *higher training error*, as reported in [11, 42] and thoroughly verified by our experiments. Fig. 1 shows a typical example.

The degradation (of training accuracy) indicates that not all systems are similarly easy to optimize. Let us consider a shallower architecture and its deeper counterpart that adds more layers onto it. There exists a solution *by construction* to the deeper model: the added layers are *identity* mapping, and the other layers are copied from the learned shallower model. The existence of this constructed solution indicates that a deeper model should produce no higher training error than its shallower counterpart. But experiments show that our current solvers on hand are unable to find solutions that

用于图像识别的深度残差学习

Kaiming He Xiangyu Zhang Shaoqing Ren Jian Sun 微软研究院
kahe, v-xiangz, v-shren, jiansun}@microsoft.com

摘要

深层神经网络更难训练。我们提出了一种残差学习框架，以简化对比以往网络深得多的网络的训练。我们将网络层明确地重新表述为相对于层输入学习残差函数，而不是学习不参照输入的函数。我们提供了全面的实验证据，表明这些残差网络更易于优化，并且可以通过显著增加网络深度来提升准确率。在 ImageNet 数据集上，我们评估了深度最高达 152 层的残差网络——8×比 VGG 网络 [41] 更深，但复杂度仍然更低。由这些残差网络组成的集成模型在 ImageNet 测试集上的错误率为 3.57%。这一结果使我们在 ILSVRC 2015 分类任务中获得了第 1 名。我们还对包含 100 层和 1000 层的网络进行了 CIFAR-10 分析。

表征的深度对于许多视觉识别任务都至关重要。仅凭极深的表征，我们就在 COCO 目标检测数据集上取得了 28% 的相对提升。深度残差网络是我们参加 ILSVRC & COCO 2015 竞赛的基础¹，我们还在 ImageNet 检测、ImageNet 定位、COCO 检测和 COCO 分割任务中均夺得了第一名。

1. 引言

深度卷积神经网络 [22, 21] 推动图像分类领域取得了一系列突破 [21, 50, 40]。深度网络以端到端的多层方式，自然地整合低、中、高层特征 [50] 与分类器；堆叠层数（深度）越多，特征的“层次”也越丰富。近期研究证据[41, 44] 表明，网络深度至关重要；在具有挑战性的 ImageNet 数据集 [36] 上，领先成果 [41, 44, 13, 16] 均采用了深度为十六 [41] 至三十 [16] 层的“超深” [41] 模型。许多其他复杂的视觉识别任务 [8, 12, 7, 32, 27] 也

¹<http://image-net.org/challenges/LSVRC/2015/> 以及 <http://mscoco.org/dataset/#detections-challenge2015>。

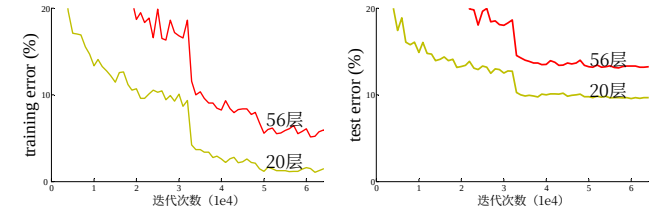


图 1. CIFAR-10 上 20 层和 56 层“普通”网络的训练误差（左）和测试误差（右）。更深的网络训练误差更高，因此测试误差也更高。图 4 展示了 ImageNet 上的类似现象。

从超深模型中获益良多。

深度如此重要，由此引出一个问题：学习更好的网络，是否只需堆叠更多层？回答这个问题的一大障碍，是臭名昭著的梯度消失/爆炸问题 [1, 9]，它从一开始就阻碍收敛。然而，通过归一化初始化 [23, 9, 37, 13] 和中间归一化层[16]，这一问题在很大程度上得到了解决，使得包含数十层的网络能够在反向传播 [22] 的随机梯度下降（SGD）中开始收敛。

当更深的网络能够开始收敛时，一个退化问题便暴露出来：随着网络深度增加，准确率达到饱和（这或许并不令人意外），随后迅速下降。令人意外的是，这种退化并非由过拟合导致；正如 [11, 42] 所报告的，并且我们的实验对此进行了充分验证，向足够深的模型添加更多层会导致更高的训练误差。图 1 展示了一个典型示例。

（训练准确率的）退化表明，并非所有系统都同样容易优化。考虑一个较浅的架构及其更深的对应架构，后者是在前者基础上增加了更多层。对于更深的模型，存在一个按构造得到的解：新增层是恒等映射，其余层则从已训练的较浅模型中复制。这个构造解的存在表明，更深的模型训练误差不应高于其较浅的对应模型。但实验表明，我们现有的求解器无法找到这样的解

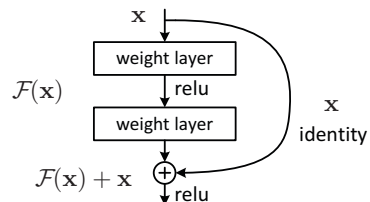


Figure 2. Residual learning: a building block.

are comparably good or better than the constructed solution (or unable to do so in feasible time).

In this paper, we address the degradation problem by introducing a *deep residual learning* framework. Instead of hoping each few stacked layers directly fit a desired underlying mapping, we explicitly let these layers fit a residual mapping. Formally, denoting the desired underlying mapping as $\mathcal{H}(\mathbf{x})$, we let the stacked nonlinear layers fit another mapping of $\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$. The original mapping is recast into $\mathcal{F}(\mathbf{x}) + \mathbf{x}$. We hypothesize that it is easier to optimize the residual mapping than to optimize the original, unreferenced mapping. To the extreme, if an identity mapping were optimal, it would be easier to push the residual to zero than to fit an identity mapping by a stack of nonlinear layers.

The formulation of $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ can be realized by feedforward neural networks with “shortcut connections” (Fig. 2). Shortcut connections [2, 34, 49] are those skipping one or more layers. In our case, the shortcut connections simply perform *identity* mapping, and their outputs are added to the outputs of the stacked layers (Fig. 2). Identity shortcut connections add neither extra parameter nor computational complexity. The entire network can still be trained end-to-end by SGD with backpropagation, and can be easily implemented using common libraries (*e.g.*, Caffe [19]) without modifying the solvers.

We present comprehensive experiments on ImageNet [36] to show the degradation problem and evaluate our method. We show that: 1) Our extremely deep residual nets are easy to optimize, but the counterpart “plain” nets (that simply stack layers) exhibit higher training error when the depth increases; 2) Our deep residual nets can easily enjoy accuracy gains from greatly increased depth, producing results substantially better than previous networks.

Similar phenomena are also shown on the CIFAR-10 set [20], suggesting that the optimization difficulties and the effects of our method are not just akin to a particular dataset. We present successfully trained models on this dataset with over 100 layers, and explore models with over 1000 layers.

On the ImageNet classification dataset [36], we obtain excellent results by extremely deep residual nets. Our 152-layer residual net is the deepest network ever presented on ImageNet, while still having lower complexity than VGG nets [41]. Our ensemble has **3.57%** top-5 error on the

ImageNet *test* set, and won the *1st place in the ILSVRC 2015 classification competition*. The extremely deep representations also have excellent generalization performance on other recognition tasks, and lead us to further *win the 1st places on: ImageNet detection, ImageNet localization, COCO detection, and COCO segmentation* in ILSVRC & COCO 2015 competitions. This strong evidence shows that the residual learning principle is generic, and we expect that it is applicable in other vision and non-vision problems.

2. Related Work

Residual Representations. In image recognition, VLAD [18] is a representation that encodes by the residual vectors with respect to a dictionary, and Fisher Vector [30] can be formulated as a probabilistic version [18] of VLAD. Both of them are powerful shallow representations for image retrieval and classification [4, 48]. For vector quantization, encoding residual vectors [17] is shown to be more effective than encoding original vectors.

In low-level vision and computer graphics, for solving Partial Differential Equations (PDEs), the widely used Multigrid method [3] reformulates the system as subproblems at multiple scales, where each subproblem is responsible for the residual solution between a coarser and a finer scale. An alternative to Multigrid is hierarchical basis preconditioning [45, 46], which relies on variables that represent residual vectors between two scales. It has been shown [3, 45, 46] that these solvers converge much faster than standard solvers that are unaware of the residual nature of the solutions. These methods suggest that a good reformulation or preconditioning can simplify the optimization.

Shortcut Connections. Practices and theories that lead to shortcut connections [2, 34, 49] have been studied for a long time. An early practice of training multi-layer perceptrons (MLPs) is to add a linear layer connected from the network input to the output [34, 49]. In [44, 24], a few intermediate layers are directly connected to auxiliary classifiers for addressing vanishing/exploding gradients. The papers of [39, 38, 31, 47] propose methods for centering layer responses, gradients, and propagated errors, implemented by shortcut connections. In [44], an “inception” layer is composed of a shortcut branch and a few deeper branches.

Concurrent with our work, “highway networks” [42, 43] present shortcut connections with gating functions [15]. These gates are data-dependent and have parameters, in contrast to our identity shortcuts that are parameter-free. When a gated shortcut is “closed” (approaching zero), the layers in highway networks represent *non-residual* functions. On the contrary, our formulation always learns residual functions; our identity shortcuts are never closed, and all information is always passed through, with additional residual functions to be learned. In addition, high-

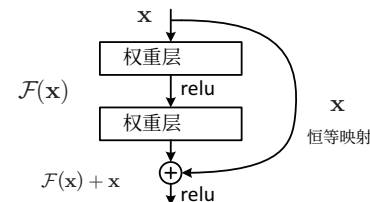


图 2. 残差学习：一个构建模块。

其效果与构造出的解相当或更好（或者无法在可行时间内做到这一点）。

本文通过引入深度残差学习框架来解决退化问题。我们不再寄希望于每几层堆叠的网络直接拟合期望的底层映射，而是明确地让这些层拟合一个残差映射。形式上，将期望的底层映射记为 $\mathcal{H}(\mathbf{x})$ ，我们让堆叠的非线性层拟合另一个映射 $\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$ 。原始映射由此改写为 $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ 。我们推测，与优化原始的、未经参照的映射相比，优化残差映射更容易。极端情况下，如果最优映射是恒等映射，那么将残差推至零，会比通过堆叠非线性层来拟合恒等映射更容易。

对 $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ 的表述可以通过带有“快捷连接”（图 2）的前馈神经网络来实现。快捷连接 [2, 34, 49] 是指跳过一层或多层的连接。在我们的例子中，快捷连接仅执行恒等映射，其输出会与堆叠层的输出相加（图 2）。恒等快捷连接既不会增加额外参数，也不会增加计算复杂度。整个网络仍可通过带反向传播的 SGD 进行端到端训练，并且可以使用常见的库（例如，Caffe [19]）轻松实现，而无需修改求解器。

我们在 ImageNet[36] 上进行了全面实验，以展示退化问题并评估我们的方法。实验表明：1) 我们的极深残差网络易于优化，而与之对应的“普通”网络（仅仅堆叠各层）在深度增加时会表现出更高的训练误差；2) 我们的深层残差网络可以轻松地大幅增加的深度中获益，提升准确率，取得显著优于以往网络的结果。

在 CIFAR-10 数据集上也观察到了类似现象[20]，这表明优化难题以及我们方法的效果并非仅限于某个特定数据集。我们成功训练出了在该数据集上深度超过 100 层的模型，并探索了深度超过 1000 层的模型。

在 ImageNet 分类数据集 [36], 上, 极深残差网络取得了出色的结果。我们的 152 层残差网络是迄今为止在 ImageNet 上提出的最深网络, 同时复杂度仍低于 VGG 网络 [41]。我们的集成模型的 top-5 错误率为**3.57%**，在该

ImageNet 测试集, 并在 *ILSVRC 2015* 分类竞赛中获得第一名。极深的表征在其他识别任务上也具有出色的泛化性能, 并使我们进一步在以下项目中夺得第一名: *ImageNet* 目标检测、*ImageNet* 定位、*COCO* 目标检测和 *COCO* 图像分割, 于 ILSVRC 和 COCO 2015 竞赛中获胜。有力证据表明, 残差学习原理具有普适性, 我们预计它适用于其他视觉及非视觉问题。

2. 相关工作

残差表示。 在图像识别领域, VLAD[18]是一种通过相对于字典的残差向量进行编码的表示方法, 而 Fisher 向量 [30] 可以表述为 [18] VLAD 的概率版本。二者都是用于图像检索和分类的强大浅层表示方法 [4, 48]。在向量量化中, 研究表明 [17] 对残差向量进行编码比对原始向量进行编码更有效。

在低层视觉和计算机图形学领域, 为求解偏微分方程 (PDE), 广泛使用的多重网格方法 [3] 会将系统重新表述为多个尺度上的子问题, 其中每个子问题负责求解较粗尺度与较细尺度之间的残差。多重网格方法的一种替代方法是层次基预条件 [45, 46], 它依赖于表示两个尺度之间残差向量的变量。研究表明 [3, 45, 46], 与不了解解的残差性质的标准求解器相比, 这些求解器的收敛速度快得多。这些方法表明, 良好的重新表述或预条件处理能够简化优化。

捷径连接。 长期以来, 人们一直在研究能够形成捷径连接的实践和理论 [2, 34, 49]。早期的一种多层感知机 (MLP) 训练方法, 是添加一个从网络输入端连接到输出端的线性层 [34, 49]。在 [44, 24], 中, 若干中间层直接连接到辅助分类器, 以解决梯度消失或爆炸问题。文献 [39, 38, 31, 47]提出了通过捷径连接实现层响应、梯度和传播误差居中处理的方法。在 [44], 中, 一个 “Inception” 层由一个捷径分支和若干更深的分支组成。

在我们开展这项工作的同时, “高速网络” [42, 43]采用带门控函数的捷径连接 [15]。这些门控依赖于数据且带有参数, 与我们不带参数的恒等捷径不同。当门控捷径 “关闭” (趋近于零) 时, 高速网络中的层表示非残差函数。相反, 我们的方案始终学习残差函数; 我们的恒等捷径从不关闭, 所有信息始终得以传递, 同时还会学习额外的残差函数。此外, 高速网

way networks have not demonstrated accuracy gains with extremely increased depth (*e.g.*, over 100 layers).

3. Deep Residual Learning

3.1. Residual Learning

Let us consider $\mathcal{H}(\mathbf{x})$ as an underlying mapping to be fit by a few stacked layers (not necessarily the entire net), with $\mathbf{x}$ denoting the inputs to the first of these layers. If one hypothesizes that multiple nonlinear layers can asymptotically approximate complicated functions², then it is equivalent to hypothesize that they can asymptotically approximate the residual functions, *i.e.*, $\mathcal{H}(\mathbf{x}) - \mathbf{x}$ (assuming that the input and output are of the same dimensions). So rather than expect stacked layers to approximate $\mathcal{H}(\mathbf{x})$, we explicitly let these layers approximate a residual function $\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$. The original function thus becomes $\mathcal{F}(\mathbf{x}) + \mathbf{x}$. Although both forms should be able to asymptotically approximate the desired functions (as hypothesized), the ease of learning might be different.

This reformulation is motivated by the counterintuitive phenomena about the degradation problem (Fig. 1, left). As we discussed in the introduction, if the added layers can be constructed as identity mappings, a deeper model should have training error no greater than its shallower counterpart. The degradation problem suggests that the solvers might have difficulties in approximating identity mappings by multiple nonlinear layers. With the residual learning reformulation, if identity mappings are optimal, the solvers may simply drive the weights of the multiple nonlinear layers toward zero to approach identity mappings.

In real cases, it is unlikely that identity mappings are optimal, but our reformulation may help to precondition the problem. If the optimal function is closer to an identity mapping than to a zero mapping, it should be easier for the solver to find the perturbations with reference to an identity mapping, than to learn the function as a new one. We show by experiments (Fig. 7) that the learned residual functions in general have small responses, suggesting that identity mappings provide reasonable preconditioning.

3.2. Identity Mapping by Shortcuts

We adopt residual learning to every few stacked layers. A building block is shown in Fig. 2. Formally, in this paper we consider a building block defined as:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}. \quad (1)$$

Here $\mathbf{x}$ and $\mathbf{y}$ are the input and output vectors of the layers considered. The function $\mathcal{F}(\mathbf{x}, \{W_i\})$ represents the residual mapping to be learned. For the example in Fig. 2 that has two layers, $\mathcal{F} = W_2\sigma(W_1\mathbf{x})$ in which σ denotes

ReLU [29] and the biases are omitted for simplifying notations. The operation $\mathcal{F} + \mathbf{x}$ is performed by a shortcut connection and element-wise addition. We adopt the second nonlinearity after the addition (*i.e.*, $\sigma(\mathbf{y})$, see Fig. 2).

The shortcut connections in Eqn.(1) introduce neither extra parameter nor computation complexity. This is not only attractive in practice but also important in our comparisons between plain and residual networks. We can fairly compare plain/residual networks that simultaneously have the same number of parameters, depth, width, and computational cost (except for the negligible element-wise addition).

The dimensions of $\mathbf{x}$ and $\mathcal{F}$ must be equal in Eqn.(1). If this is not the case (*e.g.*, when changing the input/output channels), we can perform a linear projection W_s by the shortcut connections to match the dimensions:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s\mathbf{x}. \quad (2)$$

We can also use a square matrix W_s in Eqn.(1). But we will show by experiments that the identity mapping is sufficient for addressing the degradation problem and is economical, and thus W_s is only used when matching dimensions.

The form of the residual function $\mathcal{F}$ is flexible. Experiments in this paper involve a function $\mathcal{F}$ that has two or three layers (Fig. 5), while more layers are possible. But if $\mathcal{F}$ has only a single layer, Eqn.(1) is similar to a linear layer: $\mathbf{y} = W_1\mathbf{x} + \mathbf{x}$, for which we have not observed advantages.

We also note that although the above notations are about fully-connected layers for simplicity, they are applicable to convolutional layers. The function $\mathcal{F}(\mathbf{x}, \{W_i\})$ can represent multiple convolutional layers. The element-wise addition is performed on two feature maps, channel by channel.

3.3. Network Architectures

We have tested various plain/residual nets, and have observed consistent phenomena. To provide instances for discussion, we describe two models for ImageNet as follows.

Plain Network. Our plain baselines (Fig. 3, middle) are mainly inspired by the philosophy of VGG nets [41] (Fig. 3, left). The convolutional layers mostly have 3×3 filters and follow two simple design rules: (i) for the same output feature map size, the layers have the same number of filters; and (ii) if the feature map size is halved, the number of filters is doubled so as to preserve the time complexity per layer. We perform downsampling directly by convolutional layers that have a stride of 2. The network ends with a global average pooling layer and a 1000-way fully-connected layer with softmax. The total number of weighted layers is 34 in Fig. 3 (middle).

It is worth noticing that our model has *fewer* filters and *lower* complexity than VGG nets [41] (Fig. 3, left). Our 34-layer baseline has 3.6 billion FLOPs (multiply-adds), which is only 18% of VGG-19 (19.6 billion FLOPs).

络尚未在深度大幅增加时（例如，超过100层）展现出准确率提升。

3. 深度残差学习

3.1. 残差学习

考虑将 $\mathcal{H}(\mathbf{x})$ 作为一个待由若干堆叠层拟合的底层映射（不一定是整个网络），其中 $\mathbf{x}$ 表示这些层中第一层的输入。如果假设多个非线性层能够渐近地逼近复杂函数²，那么，等价地也可以假设它们能够渐近地逼近残差函数，即即， $\mathcal{H}(\mathbf{x}) - \mathbf{x}$ （假设输入和输出的维度相同）。因此，我们不再期望堆叠层去逼近 $\mathcal{H}(\mathbf{x})$ ，而是明确地让这些层去逼近一个残差函数 $\mathcal{F}(\mathbf{x})$ ： $= \mathcal{H}(\mathbf{x}) - \mathbf{x}$ 。因此，原始函数变为 $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ 。尽管这两种形式都应当能够渐近地逼近目标函数（如前所假设），但学习的难易程度可能有所不同。

这一重新表述的动机，源于退化问题中反直觉的现象（图 1，左）。正如我们在引言中所讨论的，如果新增的层可以构造为恒等映射，那么更深的模型的训练误差应当不高于与之对应的较浅模型。退化问题表明，求解器可能难以通过多个非线性层逼近恒等映射。采用残差学习的重新表述后，如果恒等映射是最优的，求解器只需将多个非线性层的权重推向零，就可以逼近恒等映射。

在实际情况下，恒等映射不太可能是最优的，但我们的重新表述或许有助于对问题进行预条件化。如果最优函数比起零映射更接近恒等映射，那么求解器以恒等映射为参照来寻找扰动，应该比将该函数作为一个全新的函数来学习更容易。我们通过实验（图 7）表明，学习到的残差函数通常响应较小，这说明恒等映射能够提供合理的预条件。

3.2. 通过快捷连接实现恒等映射

我们每隔几个堆叠层就采用一次残差学习。图 2 展示了一个构建模块。形式上，本文考虑如下定义的构建模块：

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}. \quad (1)$$

此处， $\mathbf{x}$ 和 $\mathbf{y}$ 分别是所考虑层的输入向量和输出向量。函数 $\mathcal{F}(\mathbf{x}, \{W_i\})$ 表示需要学习的残差映射。对于图 2 中包含两层的示例， $\mathcal{F} = W_2\sigma(W_1\mathbf{x})$ 中， σ 表示

²然而，这一假设仍是一个有待解决的问题。参见 [28]。

为简化记法，省略了 ReLU [29]和偏置项。运算 $\mathcal{F} + \mathbf{x}$ 是通过快捷连接和逐元素相加实现的。我们在相加后采用第二个非线性函数（即， $\sigma(\mathbf{y})$ ，见图 2）。

式（1）中的快捷连接既不引入额外参数，也不增加计算复杂度。这不仅在实践中颇具吸引力，对于我们比较普通网络和残差网络也很重要。我们可以公平地比较参数量、深度、宽度和计算开销均相同的普通网络与残差网络（除可忽略不计的逐元素加法之外）。

式(1)中 $\mathbf{x}$ 和 $\mathcal{F}$ 的维度必须相等。如果并非如此（例如，当更改输入/输出通道时）， 我们可以通过快捷连接执行线性投影 W_s ， 以匹配维度：

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s\mathbf{x}. \quad (2)$$

我们也可以在式(1)中使用方阵 W_s 。但我们将通过实验表明，恒等映射足以解决退化问题，而且计算开销较低，因此仅在匹配维度时才使用 W_s 。

残差函数 $\mathcal{F}$ 的形式具有灵活性。本文中的实验涉及一个包含两层或三层（图5）的函数 $\mathcal{F}$ ，而层数更多也是可行的。但如果 $\mathcal{F}$ 只有一层，式(1)就类似于线性层： $\mathbf{y} = W_1\mathbf{x} + \mathbf{x}$ ，对此我们未观察到任何优势。

我们还要指出，尽管为了简洁起见，上述记号针对的是全连接层，但它们同样适用于卷积层。函数 $\mathcal{F}(\mathbf{x}, \{W_i\})$ 可表示多个卷积层。逐元素加法在两张特征图上按通道进行。

3.3. 网络架构

我们测试了各种普通网络和残差网络，并观察到了一些一致的现象。为便于讨论，下面介绍两个用于 ImageNet 的模型。

普通网络。 我们的普通网络基线（图 3，中）主要受到 VGG 网络 [41]（图 3，左）的设计理念启发。卷积层大多使用 3×3 个滤波器，并遵循两条简单的设计规则：(i) 对于相同的输出特征图尺寸，各层使用相同数量的滤波器；(ii) 如果特征图尺寸减半，则滤波器数量加倍，以保持每层的计算复杂度不变。我们通过步幅为 2 的卷积层直接进行下采样。网络以一个全局平均池化层和一个带 softmax 的 1000 类全连接层结束。图 3（中）中的网络共有 34 个带权重的层。

值得注意的是，与 VGG 网络相比，我们的模型使用的滤波器更少，且复杂度更低 [41]（图 3，左）。我们的 34 层基线模型有 36 亿次 FLOPs（乘加运算），仅为 VGG-19（196 亿次 FLOPs）的 18%。

²This hypothesis, however, is still an open question. See [28].

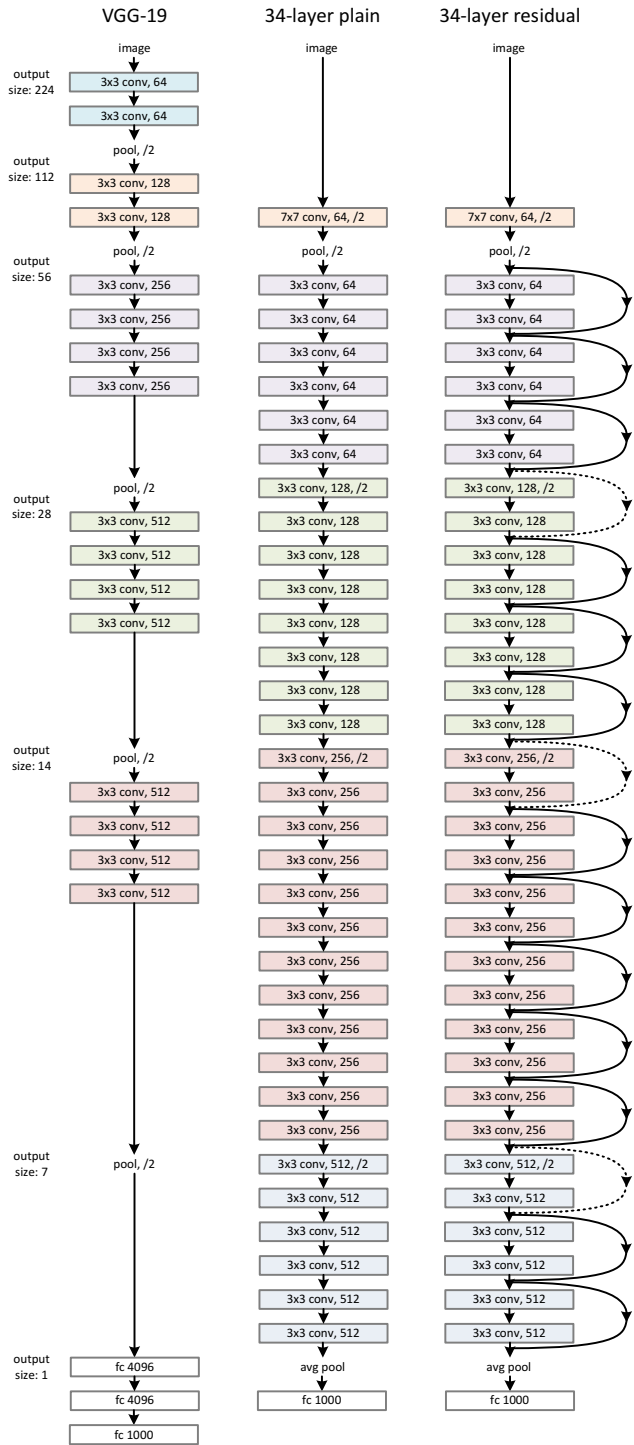


Figure 3. Example network architectures for ImageNet. **Left:** the VGG-19 model [41] (19.6 billion FLOPs) as a reference. **Middle:** a plain network with 34 parameter layers (3.6 billion FLOPs). **Right:** a residual network with 34 parameter layers (3.6 billion FLOPs). The dotted shortcuts increase dimensions. **Table 1** shows more details and other variants.

Residual Network. Based on the above plain network, we insert shortcut connections (Fig. 3, right) which turn the network into its counterpart residual version. The identity shortcuts (Eqn.(1)) can be directly used when the input and output are of the same dimensions (solid line shortcuts in Fig. 3). When the dimensions increase (dotted line shortcuts in Fig. 3), we consider two options: (A) The shortcut still performs identity mapping, with extra zero entries padded for increasing dimensions. This option introduces no extra parameter; (B) The projection shortcut in Eqn.(2) is used to match dimensions (done by 1×1 convolutions). For both options, when the shortcuts go across feature maps of two sizes, they are performed with a stride of 2.

3.4. Implementation

Our implementation for ImageNet follows the practice in [21, 41]. The image is resized with its shorter side randomly sampled in [256, 480] for scale augmentation [41]. A 224×224 crop is randomly sampled from an image or its horizontal flip, with the per-pixel mean subtracted [21]. The standard color augmentation in [21] is used. We adopt batch normalization (BN) [16] right after each convolution and before activation, following [16]. We initialize the weights as in [13] and train all plain/residual nets from scratch. We use SGD with a mini-batch size of 256. The learning rate starts from 0.1 and is divided by 10 when the error plateaus, and the models are trained for up to 60×10^4 iterations. We use a weight decay of 0.0001 and a momentum of 0.9. We do not use dropout [14], following the practice in [16].

In testing, for comparison studies we adopt the standard 10-crop testing [21]. For best results, we adopt the fully-convolutional form as in [41, 13], and average the scores at multiple scales (images are resized such that the shorter side is in $\{224, 256, 384, 480, 640\}$).

4. Experiments

4.1. ImageNet Classification

We evaluate our method on the ImageNet 2012 classification dataset [36] that consists of 1000 classes. The models are trained on the 1.28 million training images, and evaluated on the 50k validation images. We also obtain a final result on the 100k test images, reported by the test server. We evaluate both top-1 and top-5 error rates.

Plain Networks. We first evaluate 18-layer and 34-layer plain nets. The 34-layer plain net is in Fig. 3 (middle). The 18-layer plain net is of a similar form. See Table 1 for detailed architectures.

The results in Table 2 show that the deeper 34-layer plain net has higher validation error than the shallower 18-layer plain net. To reveal the reasons, in Fig. 4 (left) we compare their training/validation errors during the training procedure. We have observed the degradation problem - the

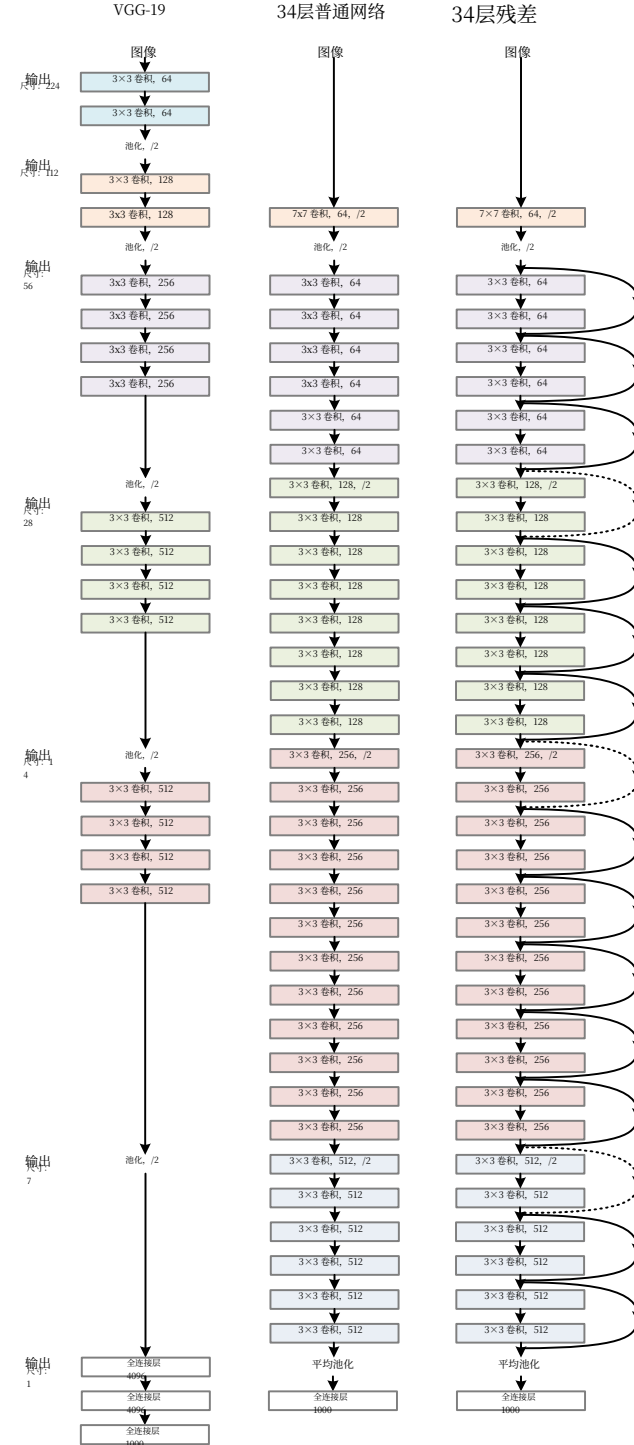


图 3. ImageNet 的网络架构示例。左侧：作为参考的 VGG-19 模型 [41]（196 亿次 FLOPs）。中间：具有 34 个参数层的普通网络（36 亿次 FLOPs）。右侧：具有 34 个参数层的残差网络（36 亿次 FLOPs）。虚线快捷连接用于增加维度。表 1 展示了更多细节和其他变体。

残差网络。 基于上述普通网络，我们加入了快捷连接（图 3，右），将网络转变为对应的残差版本。当输入和输出维度相同时，可以直接使用恒等快捷连接（式 (1)；图 3 中的实线快捷连接）。当维度增加时（图 3 中的虚线快捷连接），我们考虑两种方案：(A) 快捷连接仍执行恒等映射，并通过填充额外的零项来增加维度。该方案不会引入额外参数；(B) 使用式 (2) 中的投影快捷连接来匹配维度（通过 1×1 卷积实现）。对于这两种方案，当快捷连接跨越尺寸不同的特征图时，步幅均为 2。

3.4. 实现

我们在 ImageNet 上的实现遵循 [21, 41] 中的做法。将图像较短的一边随机缩放至 [256, 480]，以进行尺度增强 [41]。从图像或其水平翻转图像中随机裁剪一个 224×224 区域，并减去逐像素均值 [21]。采用 [21] 中的标准颜色增强方法。按照 [16] 的做法，我们在每个卷积层之后、激活函数之前使用批归一化（BN）[16]。权重按 [13] 中的方法初始化，并从头开始训练所有普通网络和残差网络。我们使用小批量大小为 256 的 SGD。学习率初始值为 0.1，在误差停滞时除以 10；模型最多训练 60×10^4 次迭代。权重衰减设为 0.0001，动量设为 0.9。遵循 [16] 中的做法，我们不使用 dropout [14]。

测试时，在对比实验中，我们采用标准的 10-crop 测试 [21]。为获得最佳结果，我们采用 [41, 13] 中的全卷积形式，并对多个尺度下的得分取平均（调整图像大小，使较短边处于 $\{224, 256, 384, 480, 640\}$ ）。

4. 实验

4.1. ImageNet 分类

我们在 ImageNet 2012 分类数据集 [36] 上评估了我们的方法，该数据集包含 1000 个类别。模型在 128 万张训练图像上进行训练，并在 5 万张验证图像上进行评估。我们还在 10 万张测试图像上获得最终结果，该结果由测试服务器报告。我们评估 top-1 和 top-5 错误率。

普通网络。 我们首先评估 18 层和 34 层普通网络。图 3（中）展示了 34 层普通网络。18 层普通网络结构类似。详细架构见表 1。

表 2 中的结果表明，更深的 34 层普通网络的验证误差高于较浅的 18 层普通网络。为揭示原因，我们在图 4（左）中比较了训练过程中它们的训练误差和验证误差。我们观察到了退化问题——该

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
		3×3 max pool, stride 2				
conv2_x	56×56	$\begin{bmatrix} 3\times 3, 64 \\ 3\times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 64 \\ 3\times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3\times 3, 128 \\ 3\times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 128 \\ 3\times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3\times 3, 256 \\ 3\times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 256 \\ 3\times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3\times 3, 512 \\ 3\times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 512 \\ 3\times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10 ⁹	3.6×10 ⁹	3.8×10 ⁹	7.6×10 ⁹	11.3×10 ⁹

Table 1. Architectures for ImageNet. Building blocks are shown in brackets (see also Fig. 5), with the numbers of blocks stacked. Down-sampling is performed by conv3_1, conv4_1, and conv5_1 with a stride of 2.

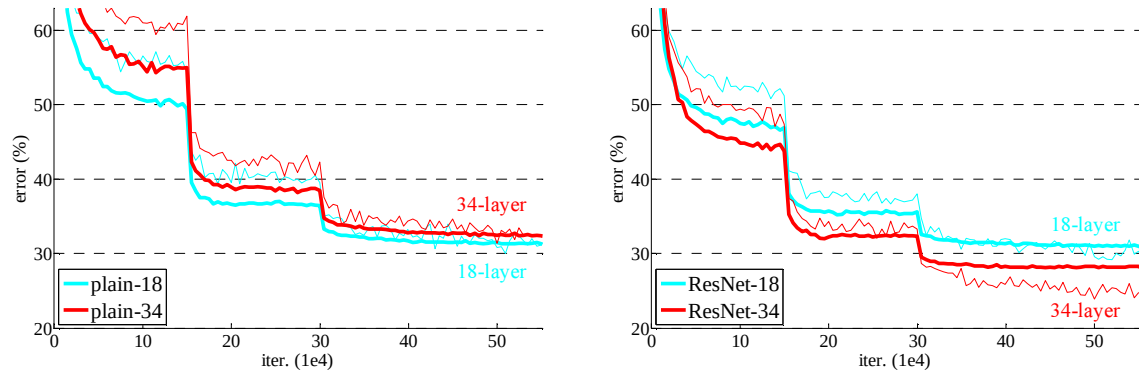


Figure 4. Training on **ImageNet**. Thin curves denote training error, and bold curves denote validation error of the center crops. Left: plain networks of 18 and 34 layers. Right: ResNets of 18 and 34 layers. In this plot, the residual networks have no extra parameter compared to their plain counterparts.

	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	25.03

Table 2. Top-1 error (% , 10-crop testing) on ImageNet validation. Here the ResNets have no extra parameter compared to their plain counterparts. Fig. 4 shows the training procedures.

34-layer plain net has higher *training* error throughout the whole training procedure, even though the solution space of the 18-layer plain network is a subspace of that of the 34-layer one.

We argue that this optimization difficulty is *unlikely* to be caused by vanishing gradients. These plain networks are trained with BN [16], which ensures forward propagated signals to have non-zero variances. We also verify that the backward propagated gradients exhibit healthy norms with BN. So neither forward nor backward signals vanish. In fact, the 34-layer plain net is still able to achieve competitive accuracy (Table 3), suggesting that the solver works to some extent. We conjecture that the deep plain nets may have exponentially low convergence rates, which impact the

reducing of the training error³. The reason for such optimization difficulties will be studied in the future.

Residual Networks. Next we evaluate 18-layer and 34-layer residual nets (*ResNets*). The baseline architectures are the same as the above plain nets, expect that a shortcut connection is added to each pair of 3×3 filters as in Fig. 3 (right). In the first comparison (Table 2 and Fig. 4 right), we use identity mapping for all shortcuts and zero-padding for increasing dimensions (option A). So they have *no extra parameter* compared to the plain counterparts.

We have three major observations from Table 2 and Fig. 4. First, the situation is reversed with residual learning – the 34-layer ResNet is better than the 18-layer ResNet (by 2.8%). More importantly, the 34-layer ResNet exhibits considerably lower training error and is generalizable to the validation data. This indicates that the degradation problem is well addressed in this setting and we manage to obtain accuracy gains from increased depth.

Second, compared to its plain counterpart, the 34-layer

³We have experimented with more training iterations (3×) and still observed the degradation problem, suggesting that this problem cannot be feasibly addressed by simply using more iterations.

层名称 输出尺寸	18层	34层	50层	101层	152层
conv1	112×112	7×7, 64, 步幅 2			
conv2 x	56×56	3×3最大池化, 步幅 2			
conv3 x	28×28	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv4 x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$
conv5 x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	平均池化、1000维全连接层、softmax			
浮点运算次数 (FLOPs)		1.8×10 ⁹	3.6×10 ⁹	3.8×10 ⁹	7.6×10 ⁹
					11.3×10 ⁹

表 1. 用于 ImageNet 的网络架构。方括号内显示了构建模块（另见图 5），括号后的数字表示堆叠的模块数量。通过步幅为 2 的 conv3 1、conv4 1 和 conv5 1 进行下采样。

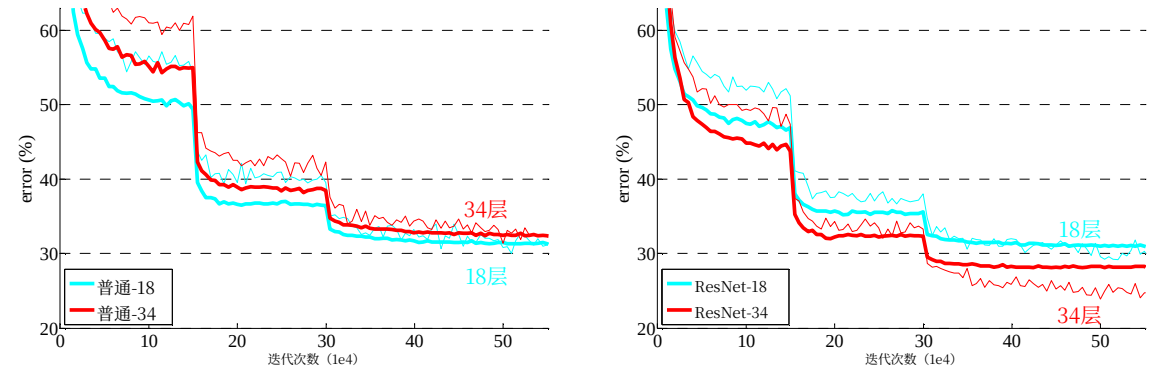


图 4. 在 **ImageNet** 上的训练。细线表示训练误差，粗线表示中心裁剪图像的验证误差。左：18 层和 34 层的普通网络。右：18 层和 34 层的 ResNet。在此图中，与对应的普通网络相比，残差网络没有增加额外参数。

	普通网络	ResNet
18 层	27.94	27.88
34 层	28.54	25.03

表 2. ImageNet 验证集上的 Top-1 错误率（%，10 裁剪测试）。这里的 ResNet 与对应的普通网络相比没有额外参数。图 4 展示了训练过程。

34层普通网络在整个训练过程中始终有更高的训练误差，尽管18层普通网络的解空间是34层普通网络解空间的子空间。

我们认为，这种优化困难不太可能 是由梯度消失造成的。这些普通网络使用 BN [16], 进行训练，这确保前向传播的信号具有非零方差。我们还验证了在使用 BN 时，反向传播的梯度具有正常的范数。因此，无论是前向信号还是反向信号都不会消失。事实上，34 层的普通网络仍能达到有竞争力的准确率（表 3），这表明优化算法在一定程度上是有效的。我们推测，深层普通网络的收敛速度可能呈指数级降低，从而影响

训练误差的降低³。这种优化困难的原因将在未来进行研究。

残差网络。 接下来，我们评估 18 层和 34 层残差网络 (*ResNet*)。基线架构与上述普通网络相同，只是在如图 3（右）所示的每对 3×3 滤波器上增加了一个捷径连接。在第一次比较中（表 2 和图 4 右），所有捷径连接都采用恒等映射，并在维度增加时使用零填充（选项 A）。因此，与对应的普通网络相比，它们没有额外参数。

从表 2 和图 4 中，我们可以得出三个主要观察结果。首先，残差学习使情况发生了逆转——34 层 ResNet 的表现优于 18 层 ResNet（高出 2.8%）。更重要的是，34 层 ResNet 的训练误差显著更低，并且能够泛化到验证数据。这表明，在这种设置下，退化问题得到了妥善解决，我们成功地通过增加深度提升了准确率。

其次，与其普通版本相比，34 层

³我们尝试了更多训练迭代次数（3×），但仍观察到了退化问题，这表明仅仅增加迭代次数并不能切实解决这一问题。

model	top-1 err.	top-5 err.
VGG-16 [41]	28.07	9.33
GoogLeNet [44]	-	9.15
PReLU-net [13]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	21.43	5.71

Table 3. Error rates (% , **10-crop** testing) on ImageNet validation. VGG-16 is based on our test. ResNet-50/101/152 are of option B that only uses projections for increasing dimensions.

method	top-1 err.	top-5 err.
VGG [41] (ILSVRC’ 14)	-	8.43 [†]
GoogLeNet [44] (ILSVRC’ 14)	-	7.89
VGG [41] (v5)	24.4	7.1
PReLU-net [13]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

Table 4. Error rates (%) of **single-model** results on the ImageNet validation set (except [†] reported on the test set).

method	top-5 err. (test)
VGG [41] (ILSVRC’ 14)	7.32
GoogLeNet [44] (ILSVRC’ 14)	6.66
VGG [41] (v5)	6.8
PReLU-net [13]	4.94
BN-inception [16]	4.82
ResNet (ILSVRC’15)	3.57

Table 5. Error rates (%) of **ensembles**. The top-5 error is on the test set of ImageNet and reported by the test server.

ResNet reduces the top-1 error by 3.5% (Table 2), resulting from the successfully reduced training error (Fig. 4 right *vs.* left). This comparison verifies the effectiveness of residual learning on extremely deep systems.

Last, we also note that the 18-layer plain/residual nets are comparably accurate (Table 2), but the 18-layer ResNet converges faster (Fig. 4 right *vs.* left). When the net is “not overly deep” (18 layers here), the current SGD solver is still able to find good solutions to the plain net. In this case, the ResNet eases the optimization by providing faster convergence at the early stage.

Identity vs. Projection Shortcuts. We have shown that

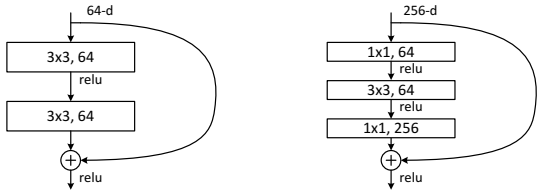


Figure 5. A deeper residual function $\mathcal{F}$ for ImageNet. Left: a building block (on 56×56 feature maps) as in Fig. 3 for ResNet-34. Right: a “bottleneck” building block for ResNet-50/101/152.

parameter-free, identity shortcuts help with training. Next we investigate projection shortcuts (Eqn.(2)). In Table 3 we compare three options: (A) zero-padding shortcuts are used for increasing dimensions, and all shortcuts are parameter-free (the same as Table 2 and Fig. 4 right); (B) projection shortcuts are used for increasing dimensions, and other shortcuts are identity; and (C) all shortcuts are projections.

Table 3 shows that all three options are considerably better than the plain counterpart. B is slightly better than A. We argue that this is because the zero-padded dimensions in A indeed have no residual learning. C is marginally better than B, and we attribute this to the extra parameters introduced by many (thirteen) projection shortcuts. But the small differences among A/B/C indicate that projection shortcuts are not essential for addressing the degradation problem. So we do not use option C in the rest of this paper, to reduce memory/time complexity and model sizes. Identity shortcuts are particularly important for not increasing the complexity of the bottleneck architectures that are introduced below.

Deeper Bottleneck Architectures. Next we describe our deeper nets for ImageNet. Because of concerns on the training time that we can afford, we modify the building block as a *bottleneck* design⁴. For each residual function $\mathcal{F}$, we use a stack of 3 layers instead of 2 (Fig. 5). The three layers are 1×1 , 3×3 , and 1×1 convolutions, where the 1×1 layers are responsible for reducing and then increasing (restoring) dimensions, leaving the 3×3 layer a bottleneck with smaller input/output dimensions. Fig. 5 shows an example, where both designs have similar time complexity.

The parameter-free identity shortcuts are particularly important for the bottleneck architectures. If the identity shortcut in Fig. 5 (right) is replaced with projection, one can show that the time complexity and model size are doubled, as the shortcut is connected to the two high-dimensional ends. So identity shortcuts lead to more efficient models for the bottleneck designs.

50-layer ResNet: We replace each 2-layer block in the

⁴Deeper *non*-bottleneck ResNets (*e.g.*, Fig. 5 left) also gain accuracy from increased depth (as shown on CIFAR-10), but are not as economical as the bottleneck ResNets. So the usage of bottleneck designs is mainly due to practical considerations. We further note that the degradation problem of plain nets is also witnessed for the bottleneck designs.

模型	Top-1错误率	Top-5错误率
VGG-16 [41]	28.07	9.33
GoogLeNet [44]	-	9.15
PReLU-net [13]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	21.43	5.71

表 3. ImageNet 验证集上的错误率 (% , **10 裁剪测试**)。VGG-16 的结果基于我们的测试。ResNet-50/101/152 采用方案 B, 即仅在增加维度时使用投影。

方法	top-1错误率	top-5错误率
VGG [41] (ILSVRC’ 14)	-	8.43 [†]
GoogLeNet [44] (ILSVRC’ 14)	-	7.89
VGG [41] (v5)	24.4	7.1
PReLU-net [13]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

表 4. ImageNet 验证集上的**单模型** 结果错误率 (%) （除 [†] 报告于测试集上的结果外）。

方法	top-5 错误率 (测试集)
VGG [41] (ILSVRC’ 14)	7.32
GoogLeNet [44](ILSVRC’ 14)	6.66
VGG [41] (v5)	6.8
PReLU-net [13]	4.94
BN-inception [16]	4.82
ResNet (ILSVRC’ 15)	3.57

表 5. **集成模型**的错误率 (%)。ImageNet 测试集上的 top-5 错误率由测试服务器报告。

ResNet 将 top-1 错误率降低了 3.5%（表 2），这得益于训练误差的成功降低（图 4 对比左图）。这一比较验证了残差学习在极深层系统中的有效性。

最后，我们还注意到，18 层的普通网络和残差网络具有相当的准确率（表 2），但 18 层 ResNet 的收敛速度更快（图 4 右图对比左图）。当网络 “深度适中”（此处为 18 层）时，当前的 SGD 求解器仍能为普通网络找到良好的解。在这种情况下，ResNet 通过在早期阶段加快收敛，降低了优化难度。

恒等捷径与投影捷径。我们已经表明

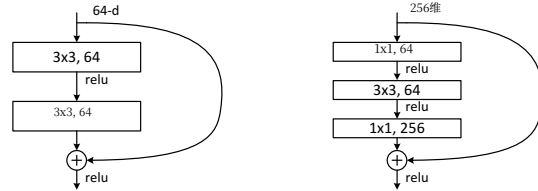


图 5. 用于 ImageNet 的更深层残差函数 $\mathcal{F}$ 。左：如图 3 所示的 ResNet-34 构建模块（基于 56×56 特征图）。右：ResNet-50/101/152 的 “瓶颈” 构建模块。

无参数的恒等捷径有助于训练。接下来，我们研究投影捷径（公式（2））。在表 3 中，我们比较了三种方案：(A) 增加维度时使用零填充捷径，且所有捷径均不含参数（与表 2 和图 4 右侧相同）；(B) 增加维度时使用投影捷径，其他捷径为恒等映射；(C) 所有捷径均为投影。

表 3 显示，这三种方案都明显优于对应的普通网络。方案 B 略优于方案 A。我们认为，这是因为方案 A 中零填充的维度确实没有进行残差学习。方案 C 略优于方案 B，我们将此归因于多个（13 个）投影捷径引入的额外参数。不过，A/B/C 之间的差异很小，这表明投影捷径并非解决退化问题的必要条件。因此，为降低内存和时间复杂度以及模型规模，本文其余部分不采用方案 C。恒等捷径对于避免增加下文介绍的瓶颈架构的复杂度尤其重要。

更深的瓶颈架构。接下来，我们介绍用于 I mageNet 的更深层网络。考虑到可承受的训练时间，我们将构建块改为瓶颈设计⁴。对于每个残差函数 $\mathcal{F}$ ，我们使用由 3 层组成的堆叠结构，而不是 2 层（图 5）。这三层分别是 1×1 、 3×3 和 1×1 卷积，其中 1×1 层负责先降低维度、再增加（恢复）维度，使 3×3 层成为输入/输出维度较小的瓶颈。图 5 展示了一个例子，两种设计的时间复杂度相近。

对于瓶颈架构，无需参数的恒等捷径尤为重要。如果将图 5（右）中的恒等捷径替换为投影，可以证明，由于捷径连接的是两个高维端，时间复杂度和模型规模都会翻倍。因此，对于瓶颈设计，恒等捷径能够构建更高效的模型。

50 层 ResNet: 我们将每个 2 层块替换为

⁴更深的非瓶颈 ResNet（例如图 5 左）也能从深度增加中获益，准确率有所提升（如 CIFAR-10 上所示），但其经济性不如瓶颈 ResNet。因此，采用瓶颈设计主要是出于实际考量。我们还注意到，瓶颈设计同样存在普通网络的退化问题。

34-layer net with this 3-layer bottleneck block, resulting in a 50-layer ResNet (Table 1). We use option B for increasing dimensions. This model has 3.8 billion FLOPs.

101-layer and 152-layer ResNets: We construct 101-layer and 152-layer ResNets by using more 3-layer blocks (Table 1). Remarkably, although the depth is significantly increased, the 152-layer ResNet (11.3 billion FLOPs) still has *lower complexity* than VGG-16/19 nets (15.3/19.6 billion FLOPs).

The 50/101/152-layer ResNets are more accurate than the 34-layer ones by considerable margins (Table 3 and 4). We do not observe the degradation problem and thus enjoy significant accuracy gains from considerably increased depth. The benefits of depth are witnessed for all evaluation metrics (Table 3 and 4).

Comparisons with State-of-the-art Methods. In Table 4 we compare with the previous best single-model results. Our baseline 34-layer ResNets have achieved very competitive accuracy. Our 152-layer ResNet has a single-model top-5 validation error of 4.49%. This single-model result outperforms all previous ensemble results (Table 5). We combine six models of different depth to form an ensemble (only with two 152-layer ones at the time of submitting). This leads to **3.57%** top-5 error on the test set (Table 5). *This entry won the 1st place in ILSVRC 2015.*

4.2. CIFAR-10 and Analysis

We conducted more studies on the CIFAR-10 dataset [20], which consists of 50k training images and 10k testing images in 10 classes. We present experiments trained on the training set and evaluated on the test set. Our focus is on the behaviors of extremely deep networks, but not on pushing the state-of-the-art results, so we intentionally use simple architectures as follows.

The plain/residual architectures follow the form in Fig. 3 (middle/right). The network inputs are 32×32 images, with the per-pixel mean subtracted. The first layer is 3×3 convolutions. Then we use a stack of $6n$ layers with 3×3 convolutions on the feature maps of sizes $\{32, 16, 8\}$ respectively, with $2n$ layers for each feature map size. The numbers of filters are $\{16, 32, 64\}$ respectively. The subsampling is performed by convolutions with a stride of 2. The network ends with a global average pooling, a 10-way fully-connected layer, and softmax. There are totally $6n+2$ stacked weighted layers. The following table summarizes the architecture:

output map size	32×32	16×16	8×8
# layers	$1+2n$	$2n$	$2n$
# filters	16	32	64

When shortcut connections are used, they are connected to the pairs of 3×3 layers (totally $3n$ shortcuts). On this dataset we use identity shortcuts in all cases (*i.e.*, option A),

method			error (%)
Maxout [10]			9.38
NIN [25]			8.81
DSN [24]			8.22
	# layers	# params	
FitNet [35]	19	2.5M	8.39
Highway [42, 43]	19	2.3M	7.54 (7.72±0.16)
Highway [42, 43]	32	1.25M	8.80
ResNet	20	0.27M	8.75
ResNet	32	0.46M	7.51
ResNet	44	0.66M	7.17
ResNet	56	0.85M	6.97
ResNet	110	1.7M	6.43 (6.61±0.16)
ResNet	1202	19.4M	7.93

Table 6. Classification error on the **CIFAR-10** test set. All methods are with data augmentation. For ResNet-110, we run it 5 times and show “best (mean±std)” as in [43].

so our residual models have exactly the same depth, width, and number of parameters as the plain counterparts.

We use a weight decay of 0.0001 and momentum of 0.9, and adopt the weight initialization in [13] and BN [16] but with no dropout. These models are trained with a mini-batch size of 128 on two GPUs. We start with a learning rate of 0.1, divide it by 10 at 32k and 48k iterations, and terminate training at 64k iterations, which is determined on a 45k/5k train/val split. We follow the simple data augmentation in [24] for training: 4 pixels are padded on each side, and a 32×32 crop is randomly sampled from the padded image or its horizontal flip. For testing, we only evaluate the single view of the original 32×32 image.

We compare $n = \{3, 5, 7, 9\}$, leading to 20, 32, 44, and 56-layer networks. Fig. 6 (left) shows the behaviors of the plain nets. The deep plain nets suffer from increased depth, and exhibit higher training error when going deeper. This phenomenon is similar to that on ImageNet (Fig. 4, left) and on MNIST (see [42]), suggesting that such an optimization difficulty is a fundamental problem.

Fig. 6 (middle) shows the behaviors of ResNets. Also similar to the ImageNet cases (Fig. 4, right), our ResNets manage to overcome the optimization difficulty and demonstrate accuracy gains when the depth increases.

We further explore $n = 18$ that leads to a 110-layer ResNet. In this case, we find that the initial learning rate of 0.1 is slightly too large to start converging⁵. So we use 0.01 to warm up the training until the training error is below 80% (about 400 iterations), and then go back to 0.1 and continue training. The rest of the learning schedule is as done previously. This 110-layer network converges well (Fig. 6, middle). It has *fewer* parameters than other deep and thin

⁵With an initial learning rate of 0.1, it starts converging (<90% error) after several epochs, but still reaches similar accuracy.

34 层网络中的这个 3 层瓶颈块，从而得到一个 50 层 ResNet（表 1）。我们使用选项 B 来增加维度。该模型有 38 亿次浮点运算（FLOPs）。

101 层和 152 层 ResNet: 我们通过使用更多的 3 层模块构建了 101 层和 152 层 ResNet（表 1）。值得注意的是，尽管深度显著增加，152 层 ResNet（113 亿 FLOPs）的复杂度仍低于 VGG-16/19 网络（153 亿/196 亿 FLOPs）。

50/101/152 层 ResNet 的准确率明显高于 34 层 ResNet（表 3 和表 4）。我们没有观察到退化问题，因此得以从大幅增加的深度中显著获益。深度带来的优势体现在所有评估指标上（表 3 和表 4）。

与最先进方法的比较。 表 4 将我们的方法与此前最佳的单模型结果进行了比较。我们的基线 34 层 ResNet 已取得极具竞争力的准确率。我们的 152 层 ResNet 单模型 top-5 验证错误率为 4.49%。这一单模型结果优于此前所有的集成结果（表 5）。我们将六个不同深度的模型组合成一个集成模型（提交时仅包含两个 152 层模型）。这使得测试集上的**top-5 错误率达到 3.57%**（表 5）。该方案获得了 ILSVRC 2015 年竞赛第一名。

4.2. CIFAR-10 与分析

我们在 CIFAR-10 数据集[20], 上进行了更多研究。该数据集包含 5 万张训练图像和 1 万张测试图像，分为 10 个类别。我们在训练集上训练模型，并在测试集上进行评估。我们的重点是研究极深网络的行为，而非追求最先进的结果，因此有意采用如下简单架构。

普通网络/残差网络的结构遵循图 3（中/右）的形式。网络输入为 32×32 图像，并减去每个像素的均值。第一层为 3×3 卷积层。随后，我们堆叠一组包含 $6n$ 层的网络，在大小分别为 $\{32, 16, 8\}$ 的特征图上使用 3×3 卷积；每种特征图大小对应 $2n$ 层。滤波器数量分别为 $\{16, 32, 64\}$ 。通过步幅为 2 的卷积进行下采样。网络最后依次采用全局平均池化、一个 10 类全连接层和 softmax。总计有 $6n+2$ 个堆叠的加权层。下表总结了该架构：

输出特征图尺寸	32×32	16×16	8×8
层数	$1+2n$	$2n$	$2n$
滤波器数量	16	32	64

使用快捷连接时，它们连接到成对的 3×3 层（总计 $3n$ 条快捷连接）。在该数据集上，所有情况下我们都使用恒等快捷连接（即，选项 A），

方法	错误率 (%)		
Maxout [10]	9.38		
NIN [25]	8.81		
DSN [24]	8.22		
	层数	参数量	
FitNet [35]	19	2.5M	8.39
Highway [42, 43]	19	2.3M	7.54 (7.72±0.16)
高速网络 [42, 43]	32	1.25M	8.80
ResNet	20	0.27M	8.75
ResNet	32	0.46M	7.51
ResNet	44	0.66M	7.17
ResNet	56	0.85M	6.97
ResNet	110	1.7M	6.43 (6.61±0.16)
ResNet	1202	19.4M	7.93

表 6. **CIFAR-10** 测试集上的分类错误率。所有方法均采用了数据增强。对于 ResNet-110，我们运行了 5 次，并按照 [43]报告“最佳值（均值±标准差）”。

因此，我们的残差模型与对应的普通模型在深度、宽度和参数数量上完全相同。

我们使用 0.0001 的权重衰减和 0.9 的动量，并采用 [13] 中的权重初始化和 BN [16]，但不使用 dropout。这些模型在两块 GPU 上以 128 的小批量进行训练。初始学习率设为 0.1，在迭代 32k 和 48k 时降至原来的十分之一，并在迭代 64k 时终止训练；这些设置是根据 45k/5k 的训练集/验证集划分确定的。训练时，我们采用 [24] 中简单的数据增强方法：在图像每侧填充 4 个像素，然后从填充后的图像或其水平翻转图像中随机采样一个 32×32 裁剪区域。测试时，我们只评估原始 32×32 图像的单一视图。

我们比较了 $n = \{3, 5, 7, 9\}$ ，由此得到 20、32、44 和 56 层的网络。图 6（左）展示了普通网络的表现。普通深层网络会受到深度增加的影响，随着网络加深，其训练误差反而更高。这一现象与 ImageNet（图 4，左）和 MNIST（见 [42]）上的情况类似，表明这种优化困难是一个根本性问题。

图 6（中）展示了 ResNet 的表现。与 ImageNet 上的情况（图 4，右）类似，我们的 ResNet 克服了优化困难，并且随着深度增加，准确率也有所提升。

我们进一步探索了 $n = 18$ ，从而得到一个 110 层的 ResNet。在这种情况下，我们发现，初始学习率 0.1 略大，不利于开始收敛⁵。因此，我们先使用 0.01 预热训练，直到训练误差低于 80%（约 400 次迭代），然后恢复为 0.1 并继续训练。其余学习率计划与之前相同。这个 110 层网络收敛良好（图 6，中间）。它拥有更少的参数，少于其他深而窄的

⁵初始学习率为 0.1 时，经过若干个 epoch 后开始收敛（误差为 <90%），但最终仍能达到相近的准确率。

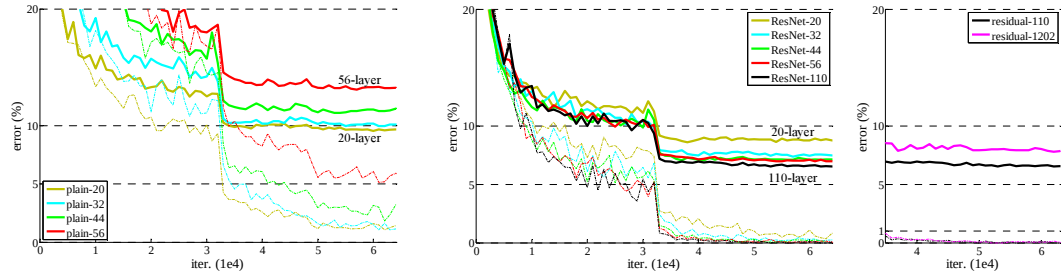


Figure 6. Training on **CIFAR-10**. Dashed lines denote training error, and bold lines denote testing error. **Left**: plain networks. The error of plain-110 is higher than 60% and not displayed. **Middle**: ResNets. **Right**: ResNets with 110 and 1202 layers.

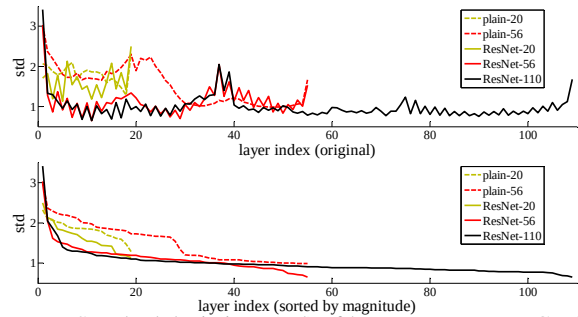


Figure 7. Standard deviations (std) of layer responses on CIFAR-10. The responses are the outputs of each 3×3 layer, after BN and before nonlinearity. **Top**: the layers are shown in their original order. **Bottom**: the responses are ranked in descending order.

networks such as FitNet [35] and Highway [42] (Table 6), yet is among the state-of-the-art results (6.43%, Table 6).

Analysis of Layer Responses. Fig. 7 shows the standard deviations (std) of the layer responses. The responses are the outputs of each 3×3 layer, after BN and before other nonlinearity (ReLU/addition). For ResNets, this analysis reveals the response strength of the residual functions. Fig. 7 shows that ResNets have generally smaller responses than their plain counterparts. These results support our basic motivation (Sec.3.1) that the residual functions might be generally closer to zero than the non-residual functions. We also notice that the deeper ResNet has smaller magnitudes of responses, as evidenced by the comparisons among ResNet-20, 56, and 110 in Fig. 7. When there are more layers, an individual layer of ResNets tends to modify the signal less.

Exploring Over 1000 layers. We explore an aggressively deep model of over 1000 layers. We set $n = 200$ that leads to a 1202-layer network, which is trained as described above. Our method shows *no optimization difficulty*, and this 10^3 -layer network is able to achieve *training error* $< 0.1\%$ (Fig. 6, right). Its test error is still fairly good (7.93%, Table 6).

But there are still open problems on such aggressively deep models. The testing result of this 1202-layer network is worse than that of our 110-layer network, although both

training data	07+12	07++12
test data	VOC 07 test	VOC 12 test
VGG-16	73.2	70.4
ResNet-101	76.4	73.8

Table 7. Object detection mAP (%) on the PASCAL VOC 2007/2012 test sets using **baseline** Faster R-CNN. See also Table 10 and 11 for better results.

metric	mAP@.5	mAP@[.5, .95]
VGG-16	41.5	21.2
ResNet-101	48.4	27.2

Table 8. Object detection mAP (%) on the COCO validation set using **baseline** Faster R-CNN. See also Table 9 for better results.

have similar training error. We argue that this is because of overfitting. The 1202-layer network may be unnecessarily large (19.4M) for this small dataset. Strong regularization such as maxout [10] or dropout [14] is applied to obtain the best results ([10, 25, 24, 35]) on this dataset. In this paper, we use no maxout/dropout and just simply impose regularization via deep and thin architectures by design, without distracting from the focus on the difficulties of optimization. But combining with stronger regularization may improve results, which we will study in the future.

4.3. Object Detection on PASCAL and MS COCO

Our method has good generalization performance on other recognition tasks. Table 7 and 8 show the object detection baseline results on PASCAL VOC 2007 and 2012 [5] and COCO [26]. We adopt *Faster R-CNN* [32] as the detection method. Here we are interested in the improvements of replacing VGG-16 [41] with ResNet-101. The detection implementation (see appendix) of using both models is the same, so the gains can only be attributed to better networks. Most remarkably, on the challenging COCO dataset we obtain a 6.0% increase in COCO’s standard metric (mAP@[.5, .95]), which is a 28% relative improvement. This gain is solely due to the learned representations.

Based on deep residual nets, we won the 1st places in several tracks in ILSVRC & COCO 2015 competitions: ImageNet detection, ImageNet localization, COCO detection, and COCO segmentation. The details are in the appendix.

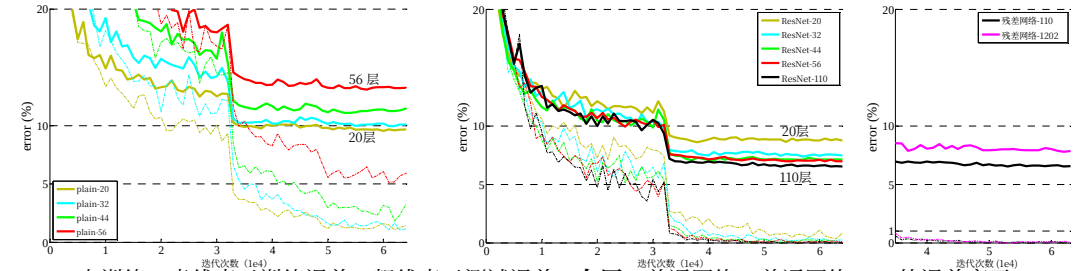


图 6. 在 **CIFAR-10** 上训练。虚线表示训练误差，粗线表示测试误差。左图：普通网络。普通网络-110 的误差高于 60%，因此未显示。中图：ResNet。右图：具有 110 层和 1202 层的 ResNet。

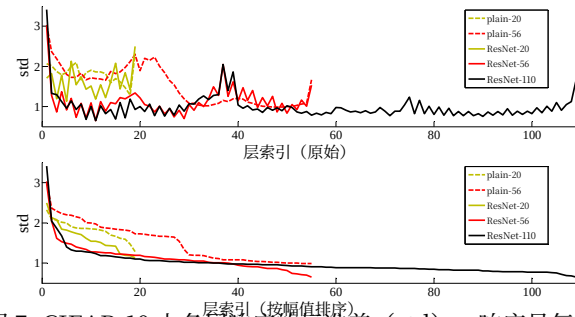


图 7. CIFAR-10 上各层响应的标准差 (std)。响应是每个 3×3 层在 BN 之后、非线性变换之前的输出。上图：各层按原始顺序排列。下图：各层响应按降序排列。

网络，例如 FitNet [35]和 Highway [42] (表 6)，但其结果仍属当前最先进水平 (6.43%，表 6)。

层响应分析。图 7 展示了层响应的标准差 (std)。响应是每个 3×3 层在 BN 之后、其他非线性变换 (ReLU/相加) 之前的输出。对于 ResNet，这项分析揭示了残差函数的响应强度。图 7 显示，ResNet 的响应通常小于对应的普通网络。这些结果支持了我们的基本动机 (第 3.1 节)：残差函数可能通常比非残差函数更接近于零。我们还注意到，更深的 ResNet 具有更小的响应幅度，图 7 中 ResNet-20、56 和 110 的对比印证了这一点。层数越多，ResNet 中的单个层对信号的改变往往越小。

探索超过 1000 层的网络。我们探索了一个深度超过 1000 层的模型。我们设定 $n = 200$ ，由此得到一个 1202 层网络，并按照上述方法进行训练。我们的方法不存在优化困难，而且这个 10^3 层网络能够达到训练误差 $< 0.1\%$ (图 6 右)。其测试误差仍相当不错 (7.93%，表 6)。

对于这种极深的模型，仍有一些尚未解决的问题。这个 1202 层网络的测试结果比我们的 110 层网络更差，尽管两者

训练数据	07+12	07++12
测试数据	VOC 07 测试集	VOC 12 测试集
VGG-16	73.2	70.4
ResNet-101	76.4	73.8

表 7. 使用**基线** Faster R-CNN 在 PASCAL VOC 2007/2012 测试集上的目标检测 mAP (%)。另请参见表 10 和表 11，了解更好的结果。

指标	mAP@.5	mAP@[.5, .95]
VGG-16	41.5	21.2
ResNet-101	48.4	27.2

表 8. 使用**基线** Faster R-CNN 在 COCO 验证集上的目标检测 mAP (%)。另请参见表 9，其中结果更好。

的训练误差相近。我们认为这是过拟合所致。对于这个小型数据集，1202 层网络可能大得没有必要 (19.4M)。采用 maxout [10] 或 dropout [14] 等强正则化方法，可以在该数据集上取得最佳结果 ([10, 25, 24, 35])。本文不使用 maxout/dropout，而是仅通过有意设计的深而窄的架构来施加正则化，从而不偏离对优化难点的关注。不过，结合更强的正则化可能会改善结果，这将留待未来研究。

4.3. PASCAL 和 MS COCO 上的目标检测

我们的方法在其他识别任务上也具有良好的泛化性能。表 7 和表 8 展示了在 PASCAL VOC 2007 和 2012 [5] 以及 COCO [26] 上的目标检测基线结果。我们采用 *Faster R-CNN* [32] 作为检测方法。这里我们关注的是将 VGG-16 [41] 替换为 ResNet-101 所带来的改进。两种模型的检测实现 (见附录) 相同，因此性能提升只能归因于更好的网络。尤为显著的是，在具有挑战性的 COCO 数据集上，我们在 COCO 的标准指标 (mAP@[.5, .95]) 上提升了 6.0%，相对提升幅度达 28%。这一提升完全得益于学得的表现。

基于深度残差网络，我们在 ILSVRC 和 COCO 2015 竞赛的多个赛道中获得了第一名：ImageNet 检测、ImageNet 定位、COCO 检测和 COCO 分割。详情见附录。

References

- [1] Y. Bengio, P. Simard, and P. Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994.
- [2] C. M. Bishop. *Neural networks for pattern recognition*. Oxford university press, 1995.
- [3] W. L. Briggs, S. F. McCormick, et al. *A Multigrid Tutorial*. Siam, 2000.
- [4] K. Chatfield, V. Lempitsky, A. Vedaldi, and A. Zisserman. The devil is in the details: an evaluation of recent feature encoding methods. In *BMVC*, 2011.
- [5] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman. The Pascal Visual Object Classes (VOC) Challenge. *IJCV*, pages 303–338, 2010.
- [6] S. Gidaris and N. Komodakis. Object detection via a multi-region & semantic segmentation-aware cnn model. In *ICCV*, 2015.
- [7] R. Girshick. Fast R-CNN. In *ICCV*, 2015.
- [8] R. Girshick, J. Donahue, T. Darrell, and J. Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *CVPR*, 2014.
- [9] X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In *AISTATS*, 2010.
- [10] I. J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, and Y. Bengio. Maxout networks. *arXiv:1302.4389*, 2013.
- [11] K. He and J. Sun. Convolutional neural networks at constrained time cost. In *CVPR*, 2015.
- [12] K. He, X. Zhang, S. Ren, and J. Sun. Spatial pyramid pooling in deep convolutional networks for visual recognition. In *ECCV*, 2014.
- [13] K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *ICCV*, 2015.
- [14] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv:1207.0580*, 2012.
- [15] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [16] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015.
- [17] H. Jegou, M. Douze, and C. Schmid. Product quantization for nearest neighbor search. *TPAMI*, 33, 2011.
- [18] H. Jegou, F. Perronnin, M. Douze, J. Sanchez, P. Perez, and C. Schmid. Aggregating local image descriptors into compact codes. *TPAMI*, 2012.
- [19] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell. Caffe: Convolutional architecture for fast feature embedding. *arXiv:1408.5093*, 2014.
- [20] A. Krizhevsky. Learning multiple layers of features from tiny images. *Tech Report*, 2009.
- [21] A. Krizhevsky, I. Sutskever, and G. Hinton. Imagenet classification with deep convolutional neural networks. In *NIPS*, 2012.
- [22] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1989.
- [23] Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller. Efficient backprop. In *Neural Networks: Tricks of the Trade*, pages 9–50. Springer, 1998.
- [24] C.-Y. Lee, S. Xie, P. Gallagher, Z. Zhang, and Z. Tu. Deeply-supervised nets. *arXiv:1409.5185*, 2014.
- [25] M. Lin, Q. Chen, and S. Yan. Network in network. *arXiv:1312.4400*, 2013.
- [26] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: Common objects in context. In *ECCV*. 2014.
- [27] J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. In *CVPR*, 2015.

- [28] G. Montúfar, R. Pascanu, K. Cho, and Y. Bengio. On the number of linear regions of deep neural networks. In *NIPS*, 2014.
- [29] V. Nair and G. E. Hinton. Rectified linear units improve restricted boltzmann machines. In *ICML*, 2010.
- [30] F. Perronnin and C. Dance. Fisher kernels on visual vocabularies for image categorization. In *CVPR*, 2007.
- [31] T. Raiko, H. Valpola, and Y. LeCun. Deep learning made easier by linear transformations in perceptrons. In *AISTATS*, 2012.
- [32] S. Ren, K. He, R. Girshick, and J. Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *NIPS*, 2015.
- [33] S. Ren, K. He, R. Girshick, X. Zhang, and J. Sun. Object detection networks on convolutional feature maps. *arXiv:1504.06066*, 2015.
- [34] B. D. Ripley. *Pattern recognition and neural networks*. Cambridge university press, 1996.
- [35] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio. Fitnets: Hints for thin deep nets. In *ICLR*, 2015.
- [36] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, et al. Imagenet large scale visual recognition challenge. *arXiv:1409.0575*, 2014.
- [37] A. M. Saxe, J. L. McClelland, and S. Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. *arXiv:1312.6120*, 2013.
- [38] N. N. Schraudolph. Accelerated gradient descent by factor-centering decomposition. Technical report, 1998.
- [39] N. N. Schraudolph. Centering neural network gradient factors. In *Neural Networks: Tricks of the Trade*, pages 207–226. Springer, 1998.
- [40] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, and Y. LeCun. Overfeat: Integrated recognition, localization and detection using convolutional networks. In *ICLR*, 2014.
- [41] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. In *ICLR*, 2015.
- [42] R. K. Srivastava, K. Greff, and J. Schmidhuber. Highway networks. *arXiv:1505.00387*, 2015.
- [43] R. K. Srivastava, K. Greff, and J. Schmidhuber. Training very deep networks. *1507.06228*, 2015.
- [44] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. In *CVPR*, 2015.
- [45] R. Szeliski. Fast surface interpolation using hierarchical basis functions. *TPAMI*, 1990.
- [46] R. Szeliski. Locally adapted hierarchical basis preconditioning. In *SIGGRAPH*, 2006.
- [47] T. Vatanen, T. Raiko, H. Valpola, and Y. LeCun. Pushing stochastic gradient towards second-order methods—backpropagation learning with transformations in nonlinearities. In *Neural Information Processing*, 2013.
- [48] A. Vedaldi and B. Fulkerson. VLFeat: An open and portable library of computer vision algorithms, 2008.
- [49] W. Venables and B. Ripley. Modern applied statistics with s-plus. 1999.
- [50] M. D. Zeiler and R. Fergus. Visualizing and understanding convolutional neural networks. In *ECCV*, 2014.

参考文献

- [1] Y. Bengio, P. Simard, and P. Frasconi. 使用梯度下降学习长期依赖关系十分困难。IEEE《神经网络汇刊》，5(2):157–166, 1994. [2] C. M. Bishop. 用于模式识别的神经网络。牛津大学出版社，1995. [3] W. L. Briggs, S. F. McCormick, et al. 多重网格教程。Siam, 2000. [4] K. Chatfield, V. Lempitsky, A. Vedaldi, and A. Zisserman. 细节决定成败：对近期特征编码方法的评估。载于 BMVC，2011. [5] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman. Pascal 视觉目标类别（VOC）挑战赛。IJCV，第 303–338 页，2010. [6] S. Gidaris and N. Komodakis. 通过多区域和语义分割感知 CNN 模型进行目标检测。载于 ICCV，2015. [7] R. Girshick. Fast R-CNN. 载于 ICCV，2015. [8] R. Girshick, J. Donahue, T. Darrell, and J. Malik. 用于精确目标检测和语义分割的丰富特征层次结构。载于 CVPR，2014. [9] X. Glorot and Y. Bengio. 理解深度前馈神经网络训练的难点。载于 AISTATS，2010. [10] I. J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, and Y. Bengio. Maxout 网络。arXiv:1302.4389，2013. [11] K. He and J. Sun. 在受限时间成本下运行卷积神经网络。载于 CVPR，2015. [12] K. He, X. Zhang, S. Ren, and J. Sun. 用于视觉识别的深度卷积网络中的空间金字塔池化。载于 ECCV，2014. [13] K. He, X. Zhang, S. Ren, and J. Sun. 深入研究整流器：在 ImageNet 分类上超越人类水平的表现。载于 ICCV，2015. [14] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov. 通过防止特征检测器的协同适应来改进神经网络。arXiv:1207.0580，2012. [15] S. Hochreiter and J. Schmidhuber. 长短期记忆。Neural Computation，9(8):1735–1780, 1997. [16] S. Ioffe and C. Szegedy. 批归一化：通过减少内部协变量偏移加速深度网络训练。载于 ICML，2015. [17] H. Jegou, M. Douze, and C. Schmid. 用于最近邻搜索的乘积量化。TPAMI，33，2011. [18] H. Jegou, F. Perronnin, M. Douze, J. Sanchez, P. Perez, and C. Schmid. 将局部图像描述符聚合为紧凑编码。TPAMI，2012. [19] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell. Caffe：用于快速特征嵌入的卷积架构。arXiv:1408.5093, 2014. [20] A. Krizhevsky. 从微小图像中学习多层特征。技术报告，2009. [21] A. Krizhevsky, I. Sutskever, and G. Hinton. 使用深度卷积神经网络进行 ImageNet 分类。载于 NIPS，2012. [22] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. 将反向传播应用于手写邮政编码识别。Neural Computation，1989. [23] Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller. 高效反向传播。载于《神经网络：技巧与窍门》，第 9–50 页。Springer，1998. [24] C.-Y. Lee, S. Xie, P. Gallagher, Z. Zhang, and Z. Tu. 深度监督网络。arXiv:1409.5185，2014. [25] M. Lin, Q. Chen, and S. Yan. 网络中的网络。arXiv:1312.4400，2013. [26] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO：情境中的常见物体。载于 ECCV，2014. [27] J. Long, E. Shelhamer, and T. Darrell. 用于语义分割的全卷积网络。载于 CVPR，2015。

- [28] G. Montúfar, R. Pascanu, K. Cho 和 Y. Bengio. 深度神经网络线性区域的数量。载于 NIPS，2014 年。[29] V. Nair 和 G. E. Hinton. 修正线性单元改进受限玻尔兹曼机。载于 ICML，2010 年。[30] F. Perronnin 和 C. Dance. 用于图像分类的视觉词汇 Fisher 核。载于 CVPR，2007 年。[31] T. Raiko, H. Valpola 和 Y. LeCun. 感知机中的线性变换让深度学习更简单。载于 AISTATS，2012 年。[32] S. Ren, K. He, R. Girshick 和 J. Sun. Faster R-CNN：借助区域提议网络实现实时目标检测。载于 NIPS，2015 年。[33] S. Ren, K. He, R. Girshick, X. Zhang 和 J. Sun. 卷积特征图上的目标检测网络。arXiv:1504.06066，2015 年。[34] B. D. Ripley. 《模式识别与神经网络》。剑桥大学出版社，1996 年。[35] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta 和 Y. Bengio. FitNets：为轻量深度网络提供提示。载于 ICLR，2015 年。[36] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein 等。ImageNet 大规模视觉识别挑战赛。arXiv:1409.0575，2014 年。[37] A. M. Saxe, J. L. McClelland 和 S. Ganguli. 深度线性神经网络学习非线性动力学的精确解。arXiv:1312.6120，2013 年。[38] N. N. Schraudolph. 通过因子中心化分解加速梯度下降。技术报告，1998 年。[39] N. N. Schraudolph. 神经网络梯度因子的中心化。载于《神经网络：技巧与诀窍》，第 207–226 页。Springer，1998 年。[40] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus 和 Y. LeCun. OverFeat：使用卷积网络实现集成识别、定位和检测。载于 ICLR，2014 年。[41] K. Simonyan 和 A. Zisserman. 用于大规模图像识别的超深卷积网络。载于 ICLR，2015 年。[42] R. K. Srivastava, K. Greff 和 J. Schmidhuber. 高速公路网络。arXiv:1505.00387，2015 年。[43] R. K. Srivastava, K. Greff 和 J. Schmidhuber. 训练超深网络。1507.06228，2015 年。[44] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke 和 A. Rabinovich. 通过卷积走向更深层次。载于 CVPR，2015 年。[45] R. Szeliski. 使用分层基函数进行快速曲面插值。TPAMI，1990 年。[46] R. Szeliski. 局部自适应分层基预条件。载于 SIGGRAPH，2006 年。[47] T. Vatanen, T. Raiko, H. Valpola 和 Y. LeCun. 推动随机梯度趋近二阶方法——使用非线性变换进行反向传播学习。载于《神经信息处理》，2013 年。[48] A. Vedaldi 和 B. Fulkerson. VLFeat：一个开放且可移植的计算机视觉算法库，2008 年。[49] W. Venables 和 B. Ripley. 《现代应用统计学（S-PLUS 版）》，1999 年。[50] M. D. Zeiler 和 R. Fergus. 卷积神经网络的可视化与理解。载于 ECCV，2014 年。

A. Object Detection Baselines

In this section we introduce our detection method based on the baseline Faster R-CNN [32] system. The models are initialized by the ImageNet classification models, and then fine-tuned on the object detection data. We have experimented with ResNet-50/101 at the time of the ILSVRC & COCO 2015 detection competitions.

Unlike VGG-16 used in [32], our ResNet has no hidden fc layers. We adopt the idea of “Networks on Conv feature maps” (NoC) [33] to address this issue. We compute the full-image shared conv feature maps using those layers whose strides on the image are no greater than 16 pixels (*i.e.*, conv1, conv2_x, conv3_x, and conv4_x, totally 91 conv layers in ResNet-101; Table 1). We consider these layers as analogous to the 13 conv layers in VGG-16, and by doing so, both ResNet and VGG-16 have conv feature maps of the same total stride (16 pixels). These layers are shared by a region proposal network (RPN, generating 300 proposals) [32] and a Fast R-CNN detection network [7]. RoI pooling [7] is performed before conv5_1. On this RoI-pooled feature, all layers of conv5_x and up are adopted for each region, playing the roles of VGG-16’s fc layers. The final classification layer is replaced by two sibling layers (classification and box regression [7]).

For the usage of BN layers, after pre-training, we compute the BN statistics (means and variances) for each layer on the ImageNet training set. Then the BN layers are fixed during fine-tuning for object detection. As such, the BN layers become linear activations with constant offsets and scales, and BN statistics are not updated by fine-tuning. We fix the BN layers mainly for reducing memory consumption in Faster R-CNN training.

PASCAL VOC

Following [7, 32], for the PASCAL VOC 2007 *test* set, we use the 5k *trainval* images in VOC 2007 and 16k *trainval* images in VOC 2012 for training (“07+12”). For the PASCAL VOC 2012 *test* set, we use the 10k *trainval+test* images in VOC 2007 and 16k *trainval* images in VOC 2012 for training (“07++12”). The hyper-parameters for training Faster R-CNN are the same as in [32]. Table 7 shows the results. ResNet-101 improves the mAP by >3% over VGG-16. This gain is solely because of the improved features learned by ResNet.

MS COCO

The MS COCO dataset [26] involves 80 object categories. We evaluate the PASCAL VOC metric (mAP @ IoU = 0.5) and the standard COCO metric (mAP @ IoU = .5:.05:.95). We use the 80k images on the train set for training and the 40k images on the val set for evaluation. Our detection system for COCO is similar to that for PASCAL VOC. We train the COCO models with an 8-GPU implementation, and thus the RPN step has a mini-batch size of

8 images (*i.e.*, 1 per GPU) and the Fast R-CNN step has a mini-batch size of 16 images. The RPN step and Fast R-CNN step are both trained for 240k iterations with a learning rate of 0.001 and then for 80k iterations with 0.0001.

Table 8 shows the results on the MS COCO validation set. ResNet-101 has a 6% increase of mAP@[.5, .95] over VGG-16, which is a 28% relative improvement, solely contributed by the features learned by the better network. Remarkably, the mAP@[.5, .95]’s absolute increase (6.0%) is nearly as big as mAP@.5’s (6.9%). This suggests that a deeper network can improve both recognition and localization.

B. Object Detection Improvements

For completeness, we report the improvements made for the competitions. These improvements are based on deep features and thus should benefit from residual learning.

MS COCO

Box refinement. Our box refinement partially follows the iterative localization in [6]. In Faster R-CNN, the final output is a regressed box that is different from its proposal box. So for inference, we pool a new feature from the regressed box and obtain a new classification score and a new regressed box. We combine these 300 new predictions with the original 300 predictions. Non-maximum suppression (NMS) is applied on the union set of predicted boxes using an IoU threshold of 0.3 [8], followed by box voting [6]. Box refinement improves mAP by about 2 points (Table 9).

Global context. We combine global context in the Fast R-CNN step. Given the full-image conv feature map, we pool a feature by global Spatial Pyramid Pooling [12] (with a “single-level” pyramid) which can be implemented as “RoI” pooling using the entire image’s bounding box as the RoI. This pooled feature is fed into the post-RoI layers to obtain a global context feature. This global feature is concatenated with the original per-region feature, followed by the sibling classification and box regression layers. This new structure is trained end-to-end. Global context improves mAP@.5 by about 1 point (Table 9).

Multi-scale testing. In the above, all results are obtained by single-scale training/testing as in [32], where the image’s shorter side is $s = 600$ pixels. Multi-scale training/testing has been developed in [12, 7] by selecting a scale from a feature pyramid, and in [33] by using maxout layers. In our current implementation, we have performed multi-scale *testing* following [33]; we have not performed multi-scale training because of limited time. In addition, we have performed multi-scale testing only for the Fast R-CNN step (but not yet for the RPN step). With a trained model, we compute conv feature maps on an image pyramid, where the image’s shorter sides are $s \in \{200, 400, 600, 800, 1000\}$.

A. 目标检测基线

本节介绍一种基于基线 Faster R-CNN [32]系统的检测方法。模型以 ImageNet 分类模型为初始化权重，然后在目标检测数据上进行微调。在 2015 年 ILSVRC 和 COCO 检测竞赛期间，我们对 ResNet-50/101 进行了实验。

与在 [32], 中使用的 VGG-16 不同，我们的 ResNet 没有隐藏的全连接层。我们采用“卷积特征图上的网络”（NoC）这一思路 [33] 来解决这个问题。我们使用图像上的步幅不超过 16 像素的层，计算整幅图像共享的卷积特征图（即，conv1、conv2_x、conv3_x 和 conv4_x；在 ResNet-101 中共包含 91 个卷积层；见表 1）。我们认为这些层类似于 VGG-16 中的 13 个卷积层，这样一来，ResNet 和 VGG-16 的卷积特征图总步幅均为 16 像素。这些层由区域提议网络（RPN，生成 300 个提议）[32] 和 Fast R-CNN 检测网络 [7]共享。RoI 池化 [7] 在 conv5_1 之前执行。对于经过 RoI 池化的特征，每个区域均采用 conv5_x 及其后续的所有层，发挥 VGG-16 全连接层的作用。最终的分类层被两个并列层（分类和边界框回归 [7]）取代。

关于 BN 层的使用，预训练后，我们在 ImageNet 训练集上计算每一层的 BN 统计量（均值和方差）。随后，在目标检测的微调过程中固定 BN 层。因此，BN 层会变为带有固定偏移和缩放的线性激活，且微调不会更新 BN 统计量。固定 BN 层主要是为了减少 Faster R-CNN 训练时的内存消耗。

PASCAL VOC

按照 [7, 32], , 对于 PASCAL VOC 2007 测试集，我们使用 VOC 2007 中的 5k 张训练验证图像和 VOC 2012 中的 16k 张训练验证图像进行训练（“07+12”）。对于 PASCAL VOC 2012 测试集，我们使用 VOC 2007 中的 10k 张训练验证+测试图像和 VOC 2012 中的 16k 张训练验证图像进行训练（“07++12”）。Faster R-CNN 的训练超参数与 [32]中相同。表 7 展示了结果。ResNet-101 的 mAP 比 VGG-16 高 >3%。这一提升完全归功于 ResNet 学到的特征更优。

MS COCO

MS COCO 数据集 [26] 包含 80 个物体类别。我们采用 PASCAL VOC 指标（mAP@IoU = 0.5）和标准 COCO 指标（mAP@IoU =.5:.05:.95）进行评估。我们使用训练集中的 8 万张图像进行训练，并使用验证集中的 4 万张图像进行评估。我们的 COCO 检测系统与 PASCAL VOC 的检测系统类似。我们使用 8 GPU 实现来训练 COCO 模型，因此 RPN 阶段的 mini-batch 大小为

8 张图像（即., 每个 GPU 1 张），Fast R-CNN 阶段的 mini-batch 大小为 16 张图像。RPN 阶段和 Fast R-CNN 阶段均先以 0.001 的学习率训练 240k 次迭代，再以 0.0001 的学习率训练 80k 次迭代。

表 8 展示了 MS COCO 验证集上的结果。与 VGG-16 相比，ResNet-101 的 mAP@[.5, .95] 提高了 6%，相对提升幅度为 28%；这完全归功于性能更优的网络所学习到的特征。值得注意的是，mAP@[.5, .95] 的绝对增幅（6.0%）几乎与 mAP@.5 的增幅（6.9%）一样大。这表明，更深的网络既能提升识别能力，也能改善定位效果。

B. 目标检测改进

为完整起见，我们报告了在竞赛中取得的改进。这些改进基于深层特征，因此理应受益于残差学习。

MS COCO

边界框细化。 我们的边界框细化方法部分沿用了 [6]中的迭代定位方法。在 Faster R-CNN 中，最终输出是一个经过回归的边界框，与其候选框不同。因此，在推理时，我们从回归后的边界框中提取新的特征，并得到新的分类得分和新的回归边界框。我们将这 300 个新预测与原来的 300 个预测合并。随后，使用 IoU 阈值 0.3 [8], 对预测边界框的并集进行非极大值抑制（NMS），并进行边界框投票 [6]。边界框细化使 mAP 提高了约 2 个百分点（表 9）。

全局上下文。 我们在 Fast R-CNN 阶段融合全局上下文。给定整幅图像的卷积特征图，我们通过全局空间金字塔池化 [12]（使用“单层”金字塔）对特征进行池化；这可以通过将整幅图像的边界框作为 RoI 来执行“RoI”池化。池化后的特征被送入 RoI 后续层，以获得全局上下文特征。然后，将该全局特征与原始的逐区域特征拼接起来，再经过并列的分类层和边界框回归层。这个新结构采用端到端方式进行训练。全局上下文使 mAP@.5 提高了约 1 个点（表 9）。

多尺度测试。 以上所有结果均采用单尺度训练/测试获得，如 [32], 所示，图像的短边为 $s = 600$ 像素。[12, 7] 通过从特征金字塔中选择一个尺度实现了多尺度训练/测试，[33] 则通过使用 maxout 层实现了这一方法。在我们的当前实现中，我们按照 [33]进行了多尺度测试；由于时间有限，我们没有进行多尺度训练。此外，我们只在 Fast R-CNN 阶段进行了多尺度测试（尚未在 RPN 阶段进行）。对于训练好的模型，我们在图像金字塔上计算卷积特征图，其中图像的短边分别为 $s \in \{200, 400, 600, 800, 1000\}$ 。

training data	COCO train		COCO trainval	
test data	COCO val		COCO test-dev	
mAP	@.5	@[.5, .95]	@.5	@[.5, .95]
baseline Faster R-CNN (VGG-16)	41.5	21.2		
baseline Faster R-CNN (ResNet-101)	48.4	27.2		
+box refinement	49.9	29.9		
+context	51.1	30.0	53.3	32.2
+multi-scale testing	53.8	32.5	55.7	34.9
ensemble			59.0	37.4

Table 9. Object detection improvements on MS COCO using Faster R-CNN and ResNet-101.

system	net	data	mAP	areo	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
baseline	VGG-16	07+12	73.2	76.5	79.0	70.9	65.5	52.1	83.1	84.7	86.4	52.0	81.9	65.7	84.8	84.6	77.5	76.7	38.8	73.6	73.9	83.0	72.6
baseline	ResNet-101	07+12	76.4	79.8	80.7	76.2	68.3	55.9	85.1	85.3	89.8	56.7	87.8	69.4	88.3	88.9	80.9	78.4	41.7	78.6	79.8	85.3	72.0
baseline+++	ResNet-101	COCO+07+12	85.6	90.0	89.6	87.8	80.8	76.1	89.9	89.9	89.6	75.5	90.0	80.7	89.6	90.3	89.1	88.7	65.4	88.1	85.6	89.0	86.8

Table 10. Detection results on the PASCAL VOC 2007 test set. The baseline is the Faster R-CNN system. The system “baseline+++” include box refinement, context, and multi-scale testing in Table 9.

system	net	data	mAP	areo	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
baseline	VGG-16	07++12	70.4	84.9	79.8	74.3	53.9	49.8	77.5	75.9	88.5	45.6	77.1	55.3	86.9	81.7	80.9	79.6	40.1	72.6	60.9	81.2	61.5
baseline	ResNet-101	07++12	73.8	86.5	81.6	77.2	58.0	51.0	78.6	76.6	93.2	48.6	80.4	59.0	92.1	85.3	84.8	80.7	48.1	77.3	66.5	84.7	65.6
baseline+++	ResNet-101	COCO+07++12	83.8	92.1	88.4	84.8	75.9	71.4	86.3	87.8	94.2	66.8	89.4	69.2	93.9	91.9	90.9	89.6	67.9	88.2	76.8	90.3	80.0

Table 11. Detection results on the PASCAL VOC 2012 test set (<http://host.robots.ox.ac.uk:8080/leaderboard/displaylb.php?challengeid=11&compid=4>). The baseline is the Faster R-CNN system. The system “baseline+++” include box refinement, context, and multi-scale testing in Table 9.

We select two adjacent scales from the pyramid following [33]. RoI pooling and subsequent layers are performed on the feature maps of these two scales [33], which are merged by maxout as in [33]. Multi-scale testing improves the mAP by over 2 points (Table 9).

Using validation data. Next we use the 80k+40k trainval set for training and the 20k test-dev set for evaluation. The test-dev set has no publicly available ground truth and the result is reported by the evaluation server. Under this setting, the results are an mAP@.5 of 55.7% and an mAP@[.5, .95] of 34.9% (Table 9). This is our single-model result.

Ensemble. In Faster R-CNN, the system is designed to learn region proposals and also object classifiers, so an ensemble can be used to boost both tasks. We use an ensemble for proposing regions, and the union set of proposals are processed by an ensemble of per-region classifiers. Table 9 shows our result based on an ensemble of 3 networks. The mAP is 59.0% and 37.4% on the test-dev set. *This result won the 1st place in the detection task in COCO 2015.*

PASCAL VOC

We revisit the PASCAL VOC dataset based on the above model. With the single model on the COCO dataset (55.7% mAP@.5 in Table 9), we fine-tune this model on the PASCAL VOC sets. The improvements of box refinement, context, and multi-scale testing are also adopted. By doing so

	val2	test
GoogLeNet [44] (ILSVRC’ 14)	-	43.9
our single model (ILSVRC’ 15)	60.5	58.8
our ensemble (ILSVRC’ 15)	63.6	62.1

Table 12. Our results (mAP, %) on the ImageNet detection dataset. Our detection system is Faster R-CNN [32] with the improvements in Table 9, using ResNet-101.

we achieve 85.6% mAP on PASCAL VOC 2007 (Table 10) and 83.8% on PASCAL VOC 2012 (Table 11)⁶. The result on PASCAL VOC 2012 is 10 points higher than the previous state-of-the-art result [6].

ImageNet Detection

The ImageNet Detection (DET) task involves 200 object categories. The accuracy is evaluated by mAP@.5. Our object detection algorithm for ImageNet DET is the same as that for MS COCO in Table 9. The networks are pre-trained on the 1000-class ImageNet classification set, and are fine-tuned on the DET data. We split the validation set into two parts (val1/val2) following [8]. We fine-tune the detection models using the DET training set and the val1 set. The val2 set is used for validation. We do not use other ILSVRC 2015 data. Our single model with ResNet-101 has

⁶<http://host.robots.ox.ac.uk:8080/anonymous/30J40J.html>, submitted on 2015-11-26.

训练数据	COCO 训练集		COCO 训练验证集	
测试数据	COCO 验证集		COCO 测试开发集	
mAP	@.5	@[.5, .95]	@.5	@[.5, .95]
基线 Faster R-CNN (VGG-16)	41.5	21.2		
基线 Faster R-CNN (ResNet-101)	48.4	27.2		
+边界框优化	49.9	29.9		
+上下文	51.1	30.0	53.3	32.2
+多尺度测试	53.8	32.5	55.7	34.9
集成			59.0	37.4

表9. 使用 Faster R-CNN 和 ResNet-101 在 MS COCO 上的目标检测改进。

系统	net	data	mAP	areo	bike	bird	boat	瓶子	bus	car	cat	椅子	cow	桌子	dog	马	摩托车	人	盆栽植物	绵羊	sofa	火车	tv
基线	VGG-16	07+12	73.2	76.5	79.0	70.9	65.5	52.1	83.1	84.7	86.4	52.0	81.9	65.7	84.8	84.6	77.5	76.7	38.8	73.6	73.9	83.0	72.6
基线	ResNet-101	07+12	76.4	79.8	80.7	76.2	68.3	55.9	85.1	85.3	89.8	56.7	87.8	69.4	88.3	88.9	80.9	78.4	41.7	78.6	79.8	85.3	72.0
基线+++	ResNet-101	COCO+07+12	85.6	90.0	89.6	87.8	80.8	76.1	89.9	89.9	89.6	75.5	90.0	80.7	89.6	90.3	89.1	88.7	65.4	88.1	85.6	89.0	86.8

表 10. PASCAL VOC 2007 测试集上的检测结果。基线为 Faster R-CNN 系统。系统 “baseline+++” 包含表 9 中的边界框细化、上下文和多尺度测试。

系统	net	data	mAP	areo	bike	bird	boat	瓶子	bus	car	cat	椅子	cow	桌子	dog	马	摩托车	人	植物	羊	sofa	火车	tv
基线	VGG-16	07++12	70.4	84.9	79.8	74.3	53.9	49.8	77.5	75.9	88.5	45.6	77.1	55.3	86.9	81.7	80.9	79.6	40.1	72.6	60.9	81.2	61.5
基线	ResNet-101	07++12	73.8	86.5	81.6	77.2	58.0	51.0	78.6	76.6	93.2	48.6	80.4	59.0	92.1	85.3	84.8	80.7	48.1	77.3	66.5	84.7	65.6
基线+++	ResNet-101	COCO+07++12	83.8	92.1	88.4	84.8	75.9	71.4	86.3	87.8	94.2	66.8	89.4	69.2	93.9	91.9	90.9	89.6	67.9	88.2	76.8	90.3	80.0

表 11. PASCAL VOC 2012 测试集上的检测结果（<http://host.robots.ox.ac.uk:8080/leaderboard/displaylb.php?challengeid=11&compid=4>）。基线是 Faster R-CNN 系统。表 9 中的“基线+++”系统包含边界框优化、上下文信息和多尺度测试。

我们按照[33]从金字塔中选择两个相邻尺度。RoI 池化及后续层在这两个尺度的特征图上执行 [33].，并按照 [33]中的方式通过 maxout 合并。多尺度测试使 mAP 提高了 2 个百分点以上（表 9）。

使用验证数据。接下来，我们使用 80k+40k trainval 集进行训练，并使用 20k test-dev 集进行评估。test-dev 集没有公开的真实标注，结果由评估服务器报告。在此设置下，mAP@.5 为 55.7%，mAP@[.5, .95] 为 34.9%（表 9）。这是我们的单模型结果。集成。在 Faster R-CNN 中，系统旨在学习区域提议，同时也学习目标分类器，因此可以通过集成来提升这两项任务的表现。我们使用集成来生成区域提议，并将所有提议合并后交由一个由各区域分类器组成的集成进行处理。表 9 展示了我们使用 3 个网络组成的集成所取得的结果。在 test-dev 集上的 mAP 为 59.0% 和 37.4%。这一结果在 *COCO 2015* 检测任务中获得了第一名。

PASCAL VOC

我们基于上述模型重新评估 PASCAL VOC 数据集。我们使用在 COCO 数据集上训练的单个模型（表 9 中 mAP@.5 为 55.7%），在 PASCAL VOC 数据集上对该模型进行微调。我们还采用了边界框优化、上下文信息和多尺度测试等改进。通过这种方式

	val2	test
GoogLeNet [44] (ILSVRC’ 14)	-	43.9
我们的单模型（ILSVRC’ 15）	60.5	58.8
我们的集成模型（ILSVRC’ 15）	63.6	62.1

表 12. 我们在 ImageNet 目标检测数据集上的结果（mAP，%）。我们的检测系统是 Faster R-CNN [32]，采用表 9 中的改进，并使用 ResNet-101。

我们在 PASCAL VOC 2007 上取得了 85.6% 的 mAP（表 10），在 PASCAL VOC 2012 上取得了 83.8% 的 mAP（表 11）。在 PASCAL VOC 2012 上的结果比此前最先进的结果高出 10 个百分点 [6]。

ImageNet 目标检测

ImageNet 检测（DET）任务涉及 200 个物体类别。准确率通过 mAP@.5 进行评估。我们用于 ImageNet DET 的目标检测算法与表 9 中用于 MS COCO 的算法相同。网络先在包含 1000 个类别的 ImageNet 分类数据集上进行预训练，然后在 DET 数据上进行微调。按照 [8]，我们将验证集分为两部分（val1/val2）。我们使用 DET 训练集和 val1 集对检测模型进行微调。val2 集用于验证。我们没有使用其他 ILSVRC 2015 数据。我们的 ResNet-101 单模型在 DET 测试集上的

⁶<http://host.robots.ox.ac.uk:8080/anonymous/30J40J.html>，于 2015-11-26 提交。

LOC method	LOC network	testing	LOC error on GT CLS	classification network	top-5 LOC error on predicted CLS
VGG’s [41]	VGG-16	1-crop	33.1 [41]		
RPN	ResNet-101	1-crop	13.3		
RPN	ResNet-101	dense	11.7		
RPN	ResNet-101	dense		ResNet-101	14.4
RPN+RCNN	ResNet-101	dense		ResNet-101	10.6
RPN+RCNN	ensemble	dense		ensemble	8.9

Table 13. Localization error (%) on the ImageNet validation. In the column of “LOC error on GT class” ([41]), the ground truth class is used. In the “testing” column, “1-crop” denotes testing on a center crop of 224×224 pixels, “dense” denotes dense (fully convolutional) and multi-scale testing.

58.8% mAP and our ensemble of 3 models has 62.1% mAP on the DET test set (Table 12). *This result won the 1st place in the ImageNet detection task in ILSVRC 2015*, surpassing the second place by **8.5 points** (absolute).

C. ImageNet Localization

The ImageNet Localization (LOC) task [36] requires to classify and localize the objects. Following [40, 41], we assume that the image-level classifiers are first adopted for predicting the class labels of an image, and the localization algorithm only accounts for predicting bounding boxes based on the predicted classes. We adopt the “per-class regression” (PCR) strategy [40, 41], learning a bounding box regressor for each class. We pre-train the networks for ImageNet classification and then fine-tune them for localization. We train networks on the provided 1000-class ImageNet training set.

Our localization algorithm is based on the RPN framework of [32] with a few modifications. Unlike the way in [32] that is category-agnostic, our RPN for localization is designed in a *per-class* form. This RPN ends with two sibling 1×1 convolutional layers for binary classification (*cls*) and box regression (*reg*), as in [32]. The *cls* and *reg* layers are both in a *per-class* from, in contrast to [32]. Specifically, the *cls* layer has a 1000-d output, and each dimension is *binary logistic regression* for predicting being or not being an object class; the *reg* layer has a 1000×4-d output consisting of box regressors for 1000 classes. As in [32], our bounding box regression is with reference to multiple translation-invariant “anchor” boxes at each position.

As in our ImageNet classification training (Sec. 3.4), we randomly sample 224×224 crops for data augmentation. We use a mini-batch size of 256 images for fine-tuning. To avoid negative samples being dominate, 8 anchors are randomly sampled for each image, where the sampled positive and negative anchors have a ratio of 1:1 [32]. For testing, the network is applied on the image fully-convolutionally.

Table 13 compares the localization results. Following [41], we first perform “oracle” testing using the ground truth class as the classification prediction. VGG’s paper [41] re-

method	top-5 localization err	
	val	test
OverFeat [40] (ILSVRC’ 13)	30.0	29.9
GoogLeNet [44] (ILSVRC’ 14)	-	26.7
VGG [41] (ILSVRC’ 14)	26.9	25.3
ours (ILSVRC’ 15)	8.9	9.0

Table 14. Comparisons of localization error (%) on the ImageNet dataset with state-of-the-art methods.

ports a center-crop error of 33.1% (Table 13) using ground truth classes. Under the same setting, our RPN method using ResNet-101 net significantly reduces the center-crop error to 13.3%. This comparison demonstrates the excellent performance of our framework. With dense (fully convolutional) and multi-scale testing, our ResNet-101 has an error of 11.7% using ground truth classes. Using ResNet-101 for predicting classes (4.6% top-5 classification error, Table 4), the top-5 localization error is 14.4%.

The above results are only based on the *proposal network* (RPN) in Faster R-CNN [32]. One may use the *detection network* (Fast R-CNN [7]) in Faster R-CNN to improve the results. But we notice that on this dataset, one image usually contains a single dominate object, and the proposal regions highly overlap with each other and thus have very similar RoI-pooled features. As a result, the image-centric training of Fast R-CNN [7] generates samples of small variations, which may not be desired for stochastic training. Motivated by this, in our current experiment we use the original R-CNN [8] that is RoI-centric, in place of Fast R-CNN.

Our R-CNN implementation is as follows. We apply the per-class RPN trained as above on the training images to predict bounding boxes for the ground truth class. These predicted boxes play a role of class-dependent proposals. For each training image, the highest scored 200 proposals are extracted as training samples to train an R-CNN classifier. The image region is cropped from a proposal, warped to 224×224 pixels, and fed into the classification network as in R-CNN [8]. The outputs of this network consist of two sibling fc layers for *cls* and *reg*, also in a per-class form. This R-CNN network is fine-tuned on the training set using a mini-batch size of 256 in the RoI-centric fashion. For testing, the RPN generates the highest scored 200 proposals for each predicted class, and the R-CNN network is used to update these proposals’ scores and box positions.

This method reduces the top-5 localization error to 10.6% (Table 13). This is our single-model result on the validation set. Using an ensemble of networks for both classification and localization, we achieve a top-5 localization error of 9.0% on the test set. This number significantly outperforms the ILSVRC 14 results (Table 14), showing a 64% relative reduction of error. *This result won the 1st place in the ImageNet localization task in ILSVRC 2015*.

LOC 方法	LOC 网络	测试	定位误差在 GT CLS 上	分类网络	top-5 定位错误在预测的 CLS 上
VGG 的 [41]	VGG-16	单裁剪	33.1 [41]		
RPN	ResNet-101	单裁剪	13.3		
RPN	ResNet-101	密集	11.7		
RPN	ResNet-101	密集		ResNet-101	14.4
RPN+RCNN	ResNet-101	密集		ResNet-101	10.6
RPN+RCNN	集成	稠密		集成	8.9

表 13. ImageNet 验证集上的定位错误率 (%)。在 “GT 类别上的 LOC 错误率” 列 ([41]) 中，使用真实类别。在 “测试” 列中，“1-crop” 表示在 224×224 像素的中心裁剪图像上进行测试；“dense” 表示稠密（全卷积）和多尺度测试。

mAP 为 58.8%，由 3 个模型组成的集成模型 mAP 为 62.1%（表 12）。这一结果使我们在 *ILSVRC 2015* 的 *ImageNet* 检测任务中获得了第 1 名，以8.5 个百分点（绝对值）的优势超过排名第二的队伍。

C. ImageNet 定位

ImageNet 定位（LOC）任务 [36] 要求对目标进行分类和定位。遵循 [40, 41], , 我们假设首先采用图像级分类器来预测图像的类别标签，而定位算法只负责根据预测类别预测边界框。我们采用 “按类别回归”（PCR）策略 [40, 41], , 为每个类别学习一个边界框回归器。我们先在 ImageNet 分类任务上预训练网络，然后针对定位任务微调网络。我们使用提供的 1000 类 ImageNet 训练集训练网络。

我们的定位算法基于 [32] 的 RPN 框架，并做了一些修改。与[32] 中不区分类别的做法不同，我们的定位 RPN 采用逐类别 形式。该 RPN 最后包含两个并列的 1×1 卷积层，分别用于二元分类（分类）和边界框回归（回归），与 [32]中的做法相同。与 [32]不同，分类层和回归 层都采用逐类别 形式。具体来说，分类层的输出为 1000 维，每一维都进行二元逻辑回归，以预测是否属于某个物体类别；回归 层的输出为 1000×4维，由针对 1000 个类别的边界框回归器组成。与 [32],一样，我们的边界框回归以每个位置处的多个平移不变 “锚框” 为参照。

与 ImageNet 分类训练（第 3.4 节）一样，我们随机采样 224×224 个裁剪块进行数据增强。微调时，我们使用包含 256 张图像的小批量。为避免负样本占主导，每张图像随机采样 8 个锚点，其中正锚点与负锚点的采样比例为 1:1 [32]。测试时，网络以全卷积方式应用于图像。

表 13 对比了定位结果。接下来[41], , 我们首先使用真实类别作为分类预测结果进行 “预言机” 测试。VGG 的论文 [41] 报

方法	top-5 定位错误率	
	val	test
OverFeat [40] (ILSVRC’ 13)	30.0	29.9
GoogLeNet [44] (ILSVRC’ 14)	-	26.7
VGG [41] (ILSVRC’ 14)	26.9	25.3
我们的方法 (ILSVRC’ 15)	8.9	9.0

表 14. 在 ImageNet 数据集上与最先进方法的定位错误率 (%) 比较。

告称，使用真实类别时，中心裁剪错误率为 33.1%（表 13）。在相同设置下，我们使用 ResNet-101 网络的 RPN 方法将中心裁剪错误率大幅降低至 13.3%。这一对比展现了我们框架的出色性能。通过密集（全卷积）和多尺度测试，我们的 ResNet-101 在使用真实类别时的错误率为 11.7%。使用 ResNet-101 预测类别（top-5 分类错误率为 4.6%，见表 4）时，top-5 定位错误率为 14.4%。

以上结果仅基于 Faster R-CNN [32]中的候选区域网络（RPN）。可以使用 Faster R-CNN 中的检测网络（Fast R-CNN [7]）来改进结果。但我们注意到，在这个数据集中，一张图像通常只包含一个占主导地位的物体，而且候选区域之间高度重叠，因此 RoI 池化特征非常相似。因此，Fast R-CNN [7]以图像为中心的训练会生成变化很小的样本，这可能不利于随机训练。受此启发，在当前实验中，我们使用原始的 R-CNN [8]（以 RoI 为中心），而不是 Fast R-CNN。

我们的 R-CNN 实现如下。我们将上述训练得到的按类别划分的 RPN 应用于训练图像，为真实类别预测边界框。这些预测框充当类别相关的候选区域。对于每张训练图像，提取得分最高的 200 个候选区域作为训练样本，用于训练 R-CNN 分类器。从候选区域裁剪图像区域，将其变形为 224×224像素，并像 R-CNN [8] 中那样输入分类网络。该网络的输出由两个并列的全连接层组成，分别用于分类 和回归，同样按类别划分。这个 R-CNN 网络采用以 RoI 为中心的方式在训练集上进行微调，迷你批次大小为 256。测试时，RPN 为每个预测类别生成得分最高的 200 个候选区域，并使用 R-CNN 网络更新这些候选区域的得分和框的位置。

该方法将 top-5 定位错误率降低至10.6%（表 13）。这是我们在验证集上的单模型结果。使用网络集成进行分类和定位后，我们在测试集上的 top-5 定位错误率达到9.0%。这一结果显著优于 ILSVRC 14 的结果（表 14），错误率相对降低了 64%。这一结果使我们在 *ILSVRC 2015* 的 *ImageNet* 定位任务中荣获第一名。